

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

Diseño e implementación de manipuladores para un robot de servicio de propósito general

TESIS

Que para obtener el título de

Ingeniero Mecatrónico

P R E S E N T A N

Rafael Cuéllar Ramírez

Marco Elian Soriano Pimentel

DIRECTOR DE TESIS

Dr. Jesús Savage Carmona



Ciudad Universitaria, Cd. Mx., 2026

Índice general

1. Introducción **5**
- 1.1. Robótica de servicio doméstico . . . 6
- 1.2. Justificación . . . 6
- 1.3. Planteamiento del problema . . . 7
- 1.4. Alcance . . . 7
- 1.5. Objetivos . . . 8
  - 1.5.1. Objetivos específicos . . . 8
- 1.6. Organización del trabajo . . . 8

2. Antecedentes **10**
- 2.1. *Justina* . . . 10
- 2.2. Diseño de manipuladores robóticos . . . 14
  - 2.2.1. Definición . . . 14
  - 2.2.2. Movilidad . . . 14
  - 2.2.3. Criterio de Grashof . . . 15
  - 2.2.4. Síntesis cinemática . . . 16
  - 2.2.5. Análisis cinemático . . . 16
- 2.3. Variables de interés e instrumentación . . . 18
- 2.4. Control de manipuladores robóticos . . . 19

3. Metodología **22**
- 3.1. Identificación de necesidades . . . 23
  - 3.1.1. Perfiles de usuario . . . 23
  - 3.1.2. Recopilación de datos . . . 24
  - 3.1.3. Interpretación y organización jerárquica . . . 24
  - 3.1.4. Importancia relativa y selección . . . 24
- 3.2. Establecimiento de especificaciones objetivo . . . 25
  - 3.2.1. Traducción de necesidades a métricas . . . 25
  - 3.2.2. Benchmarking y análisis competitivo . . . 26
  - 3.2.3. Definición de valores objetivo . . . 27
- 3.3. Generación de conceptos . . . 27
  - 3.3.1. Descomposición funcional del problema . . . 27
  - 3.3.2. Búsqueda externa de soluciones . . . 28

3.3.3. Exploración sistemática y acotamiento del espacio de soluciones 28
3.3.4. Desarrollo de conceptos completos 28

A. Justina **29**
A.1. Planning 29
A.1.1. Paquete `clips_node` (Python) 29
A.1.2. Paquete `semantic_parser` 30
A.1.3. Paquete `primitive_to_clips` 31
A.1.4. Paquete `planning_tools` (C++) 31
A.1.5. Paquetes de soporte y código de depuración 31
A.2. Supervisor 32
A.2.1. Paquete `supervisor_plan` 32
A.2.2. Paquete `supervisor_execution` 32
A.2.3. Paquete `supervisor_manager` 33
A.2.4. Paquete `supervisor_system` 34
A.2.5. Paquete `surge_et_ambula` 34
A.3. Executor 34
A.3.1. Paquete `executor_manager` 34
A.3.2. Paquete `executor_worker` 35
A.3.3. Paquete `executor_tools` (Librería FSM) 35
A.3.4. Paquete `gui_controller` (Herramienta de desarrollo) 37
A.4. Knowledge 37
A.4.1. Paquete `knowledge_bank` 37
A.4.2. Paquete `knowledge_manager` 37
A.4.3. Paquete `knowledge_tools` (Librería cliente) 38
A.5. HRI 38
A.5.1. Paquete `hri_tools` (Librería cliente y launch raíz) 38
A.5.2. Paquete `human_detection` (`human_detector`) 39
A.5.3. Paquete `human_follower` 39
A.5.4. Paquete `listener` 40
A.5.5. Paquete `talker` 41
A.5.6. Paquete `gui` (Paquete ROS `justina_gui`) 41
A.6. Vision 42
A.6.1. Paquete `vision_tools` (Librería cliente y launch raíz) 42
A.6.2. Paquete `object_recognizer` (Paquetes `yolo_detect`, `owlv2_detect` y `pc_bbox_pose`) 43
A.6.3. Paquete `face_recognizer` (Paquete ROS `face_recog`) 44
A.6.4. Paquete `table_alignment` 44
A.6.5. Paquete `vacancy_detector` 45
A.7. Navigation 46
A.7.1. Paquete `navigation_tools` (Librería cliente y launch) 46
A.7.2. Paquetes de la interfaz de control (`navigation_controller`) . . 46
A.7.3. Paquete de agrupación `navigation_planner` 47

A.7.4. Paquetes de navegación sin mapa (`mapless_navigation`) 49
A.8. Manipulation 50
A.8.1. Paquete `manipulation_tools` (Librería cliente y launch) 50
A.8.2. Paquetes de planeación y control 50
A.8.3. Paquetes de soporte y dependencias (`move_it` y `trac_ik`) . . . 51
A.9. Hardware 52
A.9.1. Paquete `hardware_tools` (Librería cliente y launch raíz) 52
A.9.2. Paquetes de percepción de la computadora principal (`door_detector`, `scanner`, `kinect_ros2` y `justina_description`) 53
A.9.3. Paquetes de bajo nivel en la computadora Robotino (`Robotino`) 54

B. Protocolo de investigación 58

C. Carta de consentimiento informado 64

Capítulo 1

Introducción

En 1920, el término **robot**, derivado de la palabra eslava *robota*, que significa trabajo subordinado, fue introducido por el dramaturgo checo Karel Čapek en su obra *Rossum's Universal Robot* [1]. Sin embargo, es hasta 1961 cuando se inicia la robótica como disciplina con la invención de los brazos autónomos para asistir a la manufactura en líneas de producción con el robot *Unimate* [2]. Sus tareas se centraban en la manipulación de piezas y ensambles desde una posición fija en un entorno controlado, prácticamente estático, con la intención de relevar a los trabajadores de tareas monótonas e incrementar la producción.

Debido a esta reciente introducción e investigación en robótica, aún existen diversas definiciones de **robot**, que varían de acuerdo con el campo de aplicación y el área de investigación. Derivado del enfoque industrial inicial, las definiciones tanto de la *International Standard Organization* en el estándar ISO 8373[1] como de la *Robot Institute of America* reducen el término de robot a un manipulador [3]. Por otro lado, Bajd [4] amplía la definición, considerando la robótica contemporánea, como la rama de la ciencia que estudia a aquellos sistemas cuya principal característica es el movimiento.

Durante las últimas décadas se han diseñado y construido una gran cantidad de robots en dos ejes principales: su grado de autonomía, entendida como la habilidad de tomar decisiones basadas en una representación interna del mundo, y su capacidad de actuar en entornos complejos. Ambos relacionados directamente con su inteligencia [2].

De esta forma, la definición que resulta más útil para el desarrollo de este trabajo es la de Arkin [5], quien propone que un robot inteligente es una máquina capaz de extraer información de su ambiente y usar el conocimiento acerca de su mundo para moverse de manera segura y significativa, con un propósito específico.

[1]El estándar ISO 8373 define robot como: "Un manipulador automáticamente controlado, reprogramable, multiuso, programable en tres o más ejes, que pueden estar fijos en un lugar o movilizarse para ser usado en aplicaciones de automatización industrial"; mientras que la RIA considera a un robot como "un manipulador multifuncional reprogramable diseñado para mover materiales, partes, herramientas o dispositivos especializados a través de diversos movimientos programados para la realización de diversas tareas".

1.1. Robótica de servicio doméstico

Los avances exponenciales en robótica han abierto la posibilidad de introducir robots en nuevos entornos. El objetivo de la robótica de servicio es reducir o eliminar el trabajo físico de las personas en lugares como casas. Los dispositivos robóticos tienen el potencial de ayudar a las personas con discapacidad a moverse o brindar compañía a ancianos. Adicionalmente, los robots pueden hacer más seguros los espacios con acciones que van desde recoger objetos para evitar tropiezos hasta verificar que se haya cerrado totalmente la llave de gas [3].

La definición de esta rama más reciente complementa la de Arkin como una máquina móvil reprogramable autónoma o semiautónoma diseñada para operar en entornos dinámicos de manera segura, confiable y robusta, y ser capaz de realizar tareas específicas [3].

Bill Gates [6] vaticinó que en un hogar promedio habría robots de servicio doméstico con propósito específico encargados de cortar el césped, vigilar, limpiar, aspirar, trapear, lavar y planchar, como ya los hay en la actualidad. Pero también habría **robots humanoides de propósito general** que ayudarían a las personas a traer y llevar objetos. Este último tipo de robot tendría la capacidad de comunicarse con los miembros de la casa utilizando lenguaje natural y planear las acciones y movimientos necesarios con base en sus núcleos de acción predefinidos [3] para realizar sus funciones.

De esta forma, los robots de servicio doméstico deben contar con tres habilidades básicas: una navegación robusta, manipulación móvil y comunicación intuitiva con el usuario [7].

En 1997, a partir de avances significativos en inteligencia artificial y robótica, tales como la derrota del campeón mundial de ajedrez Gary Kaspárov a manos del *IBM Deep Blue* y el despliegue del primer sistema robótico autónomo en Marte *Sojourner*, surgió la RoboCup (Robot World Cup), una de las iniciativas con mayor relevancia a nivel internacional en el desarrollo de estos campos. Con el objetivo de definir metas a largo plazo y estandarizar plataformas de desarrollo, se coordinan anualmente las principales instituciones enfocadas en investigación alrededor del mundo para presentar sus avances [8].

Debido al éxito de este proyecto, el alcance de la RoboCup se amplió con el surgimiento de diferentes ligas. RoboCup@HOME se fundó con el objetivo de evaluar las capacidades de la tecnología emergente aplicada a la mejora de las habilidades básicas en robótica de servicio [9].

1.2. Justificación

La manipulación es un problema bien conocido en robótica. Constituye la interfaz de interacción física con objetos y personas, siendo uno de los medios más importantes de un robot para efectuar cambios en su entorno. Las tres principales funciones de una mano humana son explorar, sujetar y manipular objetos. La manipulación robótica se

centra en resolver los últimos dos, ya que el primero cae en el terreno de la háptica [10].

La interacción física presente en nuestra vida diaria cuando se realiza una manipulación, incluso en tareas simples y comunes, va más allá de tomar y colocar algún objeto. Ejemplos de estas tareas son encender la luz, extraer un libro de una estantería, abrir una puerta o cajón, empujar algún objeto, entre otros [11].

Justina es un robot de servicio diseñado en el Laboratorio de Bio-Robótica de la UNAM para operar en entornos domésticos, concebido como parte de una iniciativa para acercar la robótica a las personas y mejorar su calidad de vida [12]. Para resolver la manipulación cuenta con dos brazos de siete grados de libertad cada uno, lo que le permitiría transportar una mayor cantidad de objetos en un menor tiempo y llevar a cabo tareas más complejas.

Contar con dos brazos completamente funcionales abre la puerta a la integración de la manipulación colaborativa. Esto permitiría al robot manejar objetos de mayor tamaño, peso y complejidad, e incorporar acciones como la apertura de contenedores o la manipulación fina de objetos.

1.3. Planteamiento del problema

Los manipuladores de *Justina* exhiben deficiencias críticas que impidieron su participación en la RoboCup@HOME 2025. Los actuadores carecen de la capacidad de carga necesaria para las tareas de manipulación doméstica, los componentes estructurales presentan deformaciones permanentes por sobrecarga y el deterioro acumulado del sistema compromete su desempeño. Asimismo, aunque posee dos brazos, la manipulación colaborativa constituye un desafío técnico pendiente.

En respuesta a esta problemática, el presente trabajo plantea el rediseño e implementación integral de los subsistemas mecánico, electrónico y de software que conforman los brazos de *Justina*.

1.4. Alcance

El presente trabajo abarca el rediseño integral de los brazos manipuladores de *Justina* en tres disciplinas:

- **Mecánica:** diseño de manipuladores y efector finales, análisis por elemento finito y manufactura.
- **Electrónica:** arquitectura del sistema de control, integración de sensores y administración de energía.
- **Software:** desarrollo de nodos de control de los manipuladores y efectores finales e infraestructura para la manipulación colaborativa.

Se contempla además la validación experimental mediante pruebas de desempeño en tareas de manipulación simple en robótica de servicio.

1.5. Objetivos

Diseñar e implementar los subsistemas mecánico y electrónico de los brazos de un robot de servicio de propósito general con el fin de manipular objetos de uso común en un entorno doméstico, así como las interfaces para su integración en el *stack* de software.

1.5.1. Objetivos específicos

- Diseñar y manufacturar dos manipuladores robóticos.
- Diseñar y manufacturar dos efectores finales (EOAT) para manipular objetos comunes de uso doméstico.
- Implementar el nodo de control en ROS para resolver el modelo de la cinemática directa e inversa de la posición de los manipuladores.
- Implementar un nodo de control en ROS para tomar los objetos.
- Actualizar el paquete de ROS con la descripción del robot.

1.6. Organización del trabajo

A continuación se presenta una visión general de la estructura del presente trabajo, ofreciendo una breve descripción del contenido y propósito de cada uno de los capítulos que lo conforman.

En el Capítulo 2 se revisan los fundamentos teóricos y técnicos que sirven de base para el desarrollo del proyecto. Se abordan los temas de mecanismos y manipuladores robóticos, control de posición y trayectoria, medición de par y fuerza en las juntas, así como el uso del *Robot Operating System* (ROS) como plataforma de desarrollo.

En el Capítulo 3 se describe el proceso metodológico seguido para el diseño del sistema. Incluye el levantamiento de necesidades, la generación de especificaciones técnicas, el análisis comparativo de soluciones existentes mediante *benchmarking*, y el desarrollo del diseño conceptual. Asimismo, se detallan las herramientas utilizadas, como la matriz *Quality Function Deployment* (QFD), para la selección y evaluación de alternativas de diseño.

En el Capítulo 4 se presenta el desarrollo integral del sistema desde una perspectiva mecatrónica. El diseño mecánico comprende la estructura del manipulador y el efector final, además de los análisis por elemento finito y el proceso de manufactura. El diseño eléctrico abarca la arquitectura del sistema de control y la administración de energía. Finalmente, se detalla la instrumentación del software y la actualización de los nodos de control del robot *Justina*.

En el Capítulo 5 se describe el protocolo de validación del sistema mecánico y electrónico, así como las pruebas de manipulación implementadas para evaluar el desempeño del robot y sus resultados.

En el Capítulo 6 se presentan las conclusiones derivadas del trabajo, acompañadas de un análisis de los resultados y una discusión sobre los logros alcanzados. También se destacan las principales contribuciones del proyecto y se proponen posibles líneas de trabajo futuro que podrían continuar y mejorar el desarrollo realizado.

Capítulo 2

Antecedentes

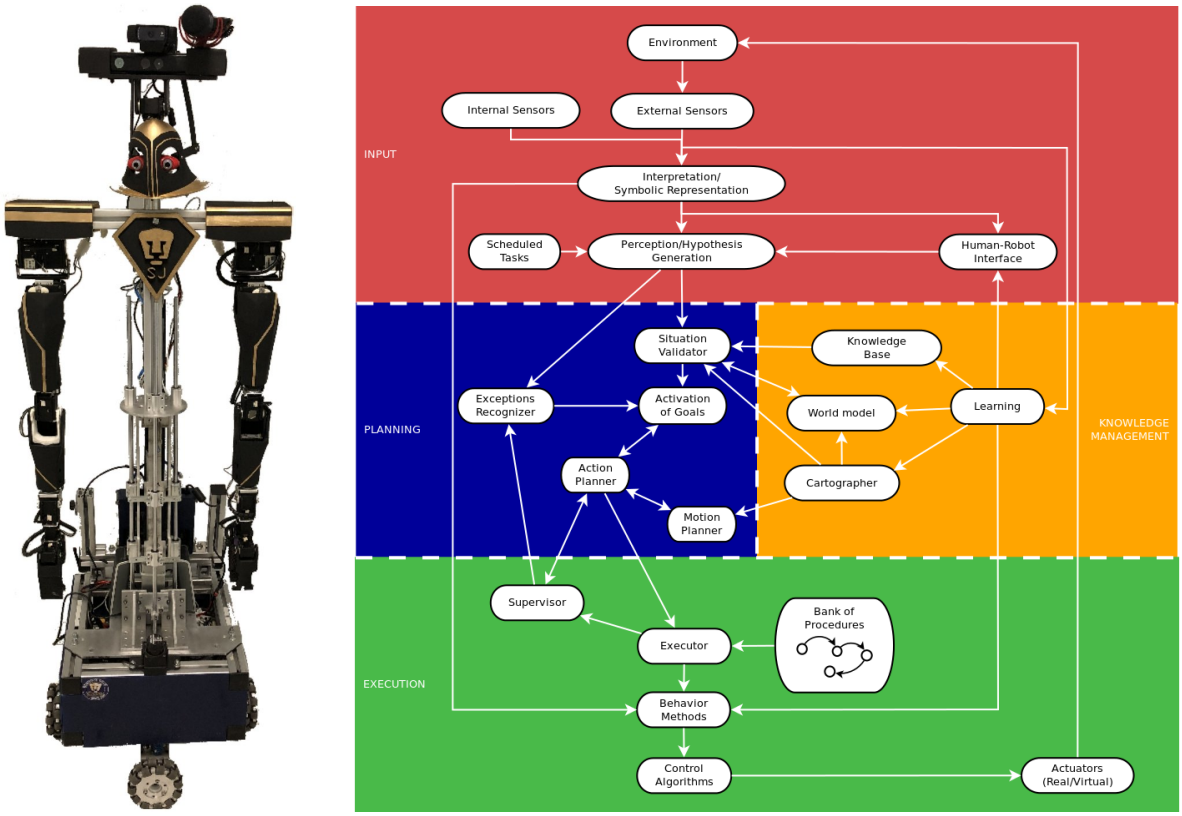
En la Universidad Nacional Autónoma de México (UNAM), el Laboratorio de Bio-Robótica de la Facultad de Ingeniería realiza investigación en áreas como robótica móvil, realidad virtual, reconocimiento de patrones, inteligencia artificial e interfaces hombre-máquina [13]. Fundado en 1996 por Jesús Savage Carmona, el laboratorio comenzó su trayectoria con la adquisición de un robot comercial estadounidense y, desde entonces, ha conformado equipos de estudiantes de licenciatura, maestría y doctorado para desarrollar prototipos propios, tales como TX8, TPR8, PAC-ITO y AL-ITA [14]. Cada plataforma ha sido fruto de una línea de trabajo orientada a la creación de robots de servicio para apoyar a las personas en sus actividades cotidianas en entornos como casas, escuelas, oficinas u hospitales [13].

Actualmente, el robot *Justina* se ha consolidado como la insignia del laboratorio y su referente en la competencia internacional RoboCup@Home. Al constituir el sistema de validación de este trabajo, en el presente capítulo se aborda su arquitectura de hardware y software, para posteriormente exponer la teoría general de manipuladores robóticos, en particular sus variables de interés, instrumentación y estrategias de control.

2.1. *Justina*

Justina (fig. 2.1a) es un robot de servicio doméstico cuyo diseño amigable e inteligente facilita la interacción con los humanos y su entorno. Como proyecto, se enmarca en una iniciativa para mejorar la calidad de vida de las personas, al tiempo que contribuye a la formación de estudiantes en ciencia e ingeniería [12]. La plataforma cuenta con una base móvil omnidireccional, un torso telescópico, un LiDAR, un micrófono cardioide, una cámara RGB-D y dos brazos manipuladores.

La integración del hardware con el software se fundamenta en la arquitectura ViRbot (fig. 2.1b), un sistema modular diseñado para coordinar la operación de robots de servicio autónomos combinando planeación de acciones con comportamientos reactivos sobre una representación dinámica del mundo [15]. Bajo este esquema, las funciones se organizan en cuatro capas: entrada, manejo del conocimiento, planeación y ejecución.



(a) Justina (b) ViRbot

Figura 2.1: Arquitectura base del robot *Justina*

Para desempeñar sus tareas, un robot de servicio basado en la arquitectura ViRbot percibe el entorno mediante sus sensores, actualiza su representación del mundo, planifica sus acciones y transmite comandos de control a sus actuadores [16]. En la implementación actual de *Justina*, este modelo conceptual se materializa empleando *Robot Operating System* (ROS), un ecosistema de código abierto cuyo *framework* proporciona herramientas que facilitan la comunicación y sincronización de procesos en el robot [17].

En ROS, las responsabilidades del sistema se estructuran en *nodos*, unidades básicas de cómputo encargadas de realizar una función específica. Los nodos pueden comunicarse entre sí utilizando diversos mecanismos, lo que permite construir sistemas distribuidos en los que sus componentes pueden ejecutarse en distintas arquitecturas de hardware e incluso en diferentes sistemas operativos [18].

A medida que las necesidades de la comunidad superaron el enfoque inicial de ROS, la plataforma evolucionó hacia ROS 2 para contemplar, desde el diseño, nuevos casos de uso, como sistemas embebidos, control en tiempo real y aplicaciones industriales [19]. Para la interacción entre nodos, ROS 2 define tres interfaces de comunicación fundamentales [20]:

- *Tópicos*: Implementan un patrón publicador/suscriptor para la transmisión con-

tinua, unidireccional y asíncrona de datos, como lecturas de sensores o estados del sistema.

- *Servicios*: Implementan un patrón de solicitud/respuesta para interacciones síncronas de corta duración, donde un cliente requiere una respuesta o confirmación inmediata por parte de un servidor.
- *Acciones*: Implementan un patrón de meta/retroalimentación/resultado para tareas asíncronas de larga duración, donde un cliente puede enviar una meta, recibir retroalimentación periódica, cancelar la ejecución y obtener un resultado.

Estas primitivas de comunicación cimentan la arquitectura actual de *Justina*, permitiendo desacoplar el sistema en módulos orquestadores y módulos especializados, cuyos paquetes, nodos e interfaces se documentan detalladamente en el anexo A. Desde una perspectiva operativa, las tareas del robot provienen de rutinas preprogramadas, como las pruebas de la competencia RoboCup@Home, o de peticiones libres derivadas de comandos de voz. En ambos casos, el procesamiento de la solicitud (fig. 2.2) recae en los módulos orquestadores:

1. *Planning*: Descompone la petición en una secuencia coherente de acciones.
2. *Supervisor*: Examina cada paso del plan generado, proporcionando rutinas de recuperación ante cualquier imprevisto.
3. *Executor*: Traduce las instrucciones de alto nivel en llamadas a los métodos de los módulos especializados correspondientes.

A partir de este punto, la ejecución se distribuye en módulos especializados que proporcionan capacidades específicas:

- *Knowledge*: Administra el estado del entorno para consulta de los demás módulos.
- *HRI*: Canaliza la interacción con personas a través del reconocimiento y síntesis de voz.
- *Vision*: Procesa imágenes para la detección y clasificación de objetos y personas.
- *Navigation*: Localiza el robot y traza rutas hacia el destino evadiendo obstáculos.
- *Manipulation*: Genera y coordina las trayectorias de los brazos y el torso.
- *Hardware*: Abstrae la comunicación con sensores y actuadores.

El desempeño de los módulos *Manipulation* y *Hardware* depende de la fidelidad con la que se representen la geometría y las relaciones cinemáticas del robot. Puesto que los brazos constituyen la interfaz principal con la que *Justina* modifica su entorno, su estudio formal resulta indispensable para traducir acciones de manipulación, como tomar y dejar objetos, en trayectorias concretas.

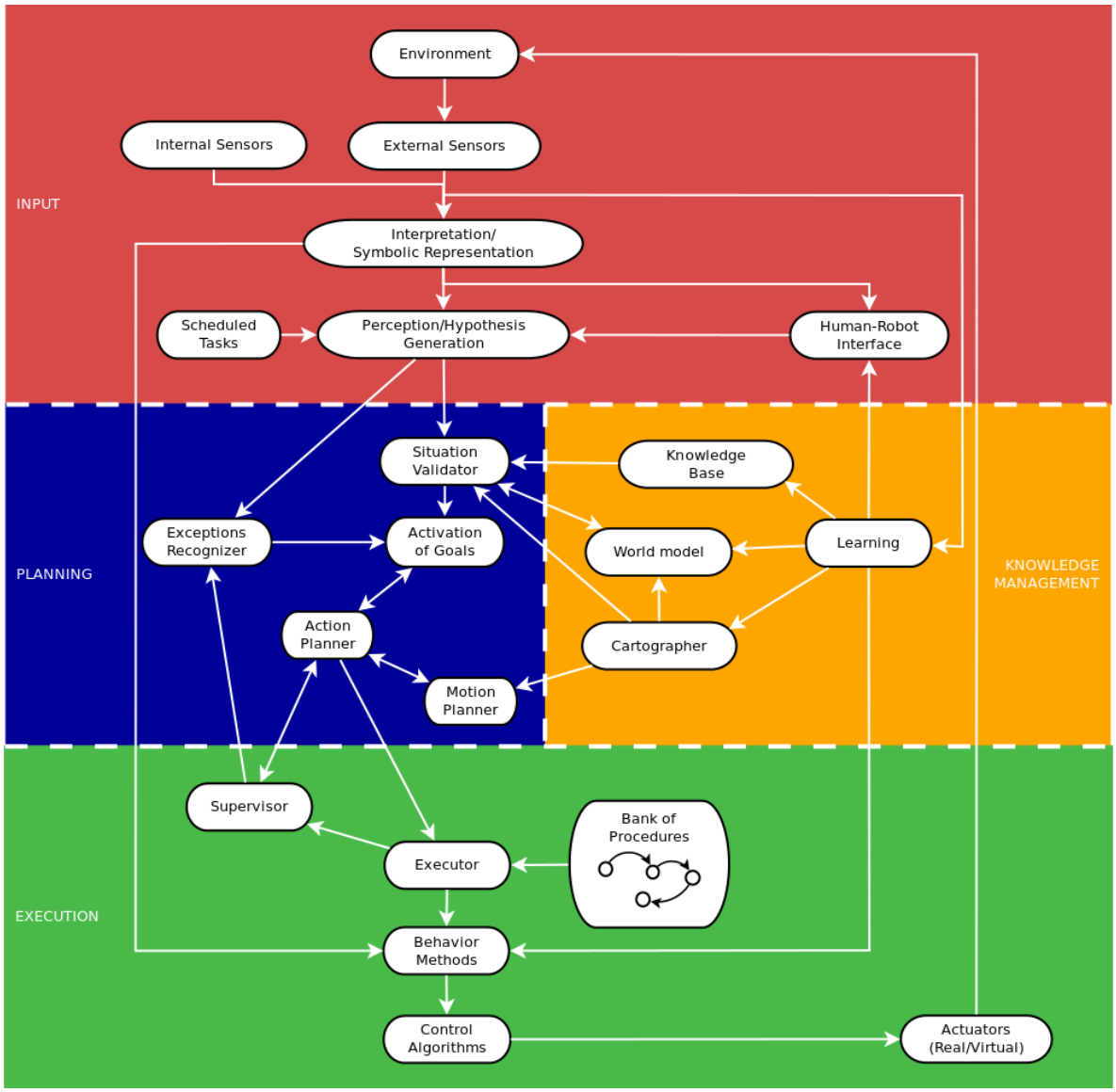


Figura 2.2: Arquitectura de operación de *Justina*.

2.2. Diseño de manipuladores robóticos

La ingeniería mecánica proporciona un conjunto de metodologías para el diseño de manipuladores robóticos, y de máquinas en general. Las ramas predominantes encargadas de ese análisis son la cinemática y la dinámica [21]. La cinemática trata el estudio del movimiento sin considerar las fuerzas que lo ocasionan. La dinámica, por otro lado, se encarga del estudio de las fuerzas sobre sistemas en movimiento. Permite definir los esfuerzos a los que estarán sometidos los elementos del sistema, como función de la aceleración y las fuerzas inerciales [22].

2.2.1. Definición

Un manipulador robótico se define como una cadena cinemática que puede programarse para realizar una amplia variedad de tareas [21]. Una cadena cinemática es un conjunto de eslabones[1] conectados por pares cinemáticos[2], para producir movimientos relativos sincronizados con un propósito determinado [21].

Desde el punto de vista de la teoría de mecanismos [22], un manipulador robótico puede considerarse un mecanismo de varios grados de libertad. A pesar de la definición de Reuleaux [23], que limita un mecanismo a cadenas cinemáticas cerradas y de un grado de libertad, para Norton [22], un mecanismo es una cadena cinemática donde al menos uno de sus eslabones se ha "aterrizado" o fijado al sistema de referencia.

2.2.2. Movilidad

El número de grados de libertad globales de un manipulador determinan su movilidad. Es el número de variables de posición independientes que deben especificarse para conocer la posición de todos los eslabones de la cadena cinemática. La condición de Gruebler considera estos factores para un mecanismo plano [22]. Sin embargo, cuando se tiene más de un eslabón conectado a tierra, el efecto que se obtiene es el de un eslabón más grande y de mayor orden. Además, se debe considerar que las semijuntas únicamente eliminan un grado de libertad en el plano. El resultado es la modificación de Kutzbach [22], mostrada en la ecuación 2.1.

$$M = 3(L-1) - 2J_1 - J_2 \tag{2.1}$$

donde:

M es el número de grados de libertad del mecanismo.

L es el número total de eslabones, incluido el eslabón fijo.

[1] Cuerpo rígido con al menos dos nodos o puntos de conexión. Se clasifican en binarios, ternarios y cuaternarios, dependiendo del número de nodos que posean [22].

[2] Unión de dos o más eslabones en sus nodos, permitiendo movimiento relativo. Se clasifican por sus grados de libertad (GDL) y las más comunes son las juntas completas (1 GDL) y las semijuntas (2 GDL) [22].

J_1 es el número de pares cinemáticos de un grado de libertad.

J_2 es el número de pares cinemáticos de dos grados de libertad.

Finalmente, para mecanismos espaciales, la modificación de Kutzbach se puede extender a eslabones con 6 GDL. De esta forma, se considera que las juntas completas anulan 5 GDL y las de orden inferior van restringiendo 1 GDL a la vez. Tomando en cuenta todos sus efectos se obtiene la ecuación 2.2, donde el subíndice indica el orden de cada tipo de junta.

$$M = 6(L-1) - 5J_1 - 4J_2 - 3J_3 - 2J_4 - J_5 \tag{2.2}$$

2.2.3. Criterio de Grashof

Para el diseño de mecanismos de cuatro barras es necesario determinar qué configuraciones permiten obtener el movimiento requerido. En este sentido, la ley de Grashof establece que para que algún eslabón tenga rotación continua *"la suma de las longitudes del eslabón más corto (s) y más largo (l) no puede ser más grande que la suma de las longitudes de los eslabones restantes (p y q)"* [24], tal como se expresa en la inecuación 2.3.

$$s + l \leq p + q \tag{2.3}$$

Derivado de la condición de Grashof se puede agrupar a las cadenas cinemáticas de cuatro barras planas en tres clases, según se muestra en la figura 2.3. La Clase I cumple con la desigualdad 2.3 y sus inversiones[3] serán la manivela-balancín, la doble manivela y un doble balancín de Grashof, donde el eslabón acoplador hace una revolución completa. La Clase II no cumple con la condición de Grashof y todas sus inversiones serán balancines triples. La Clase III, o caso especial de Grashof, es cuando se cumple la igualdad [22].

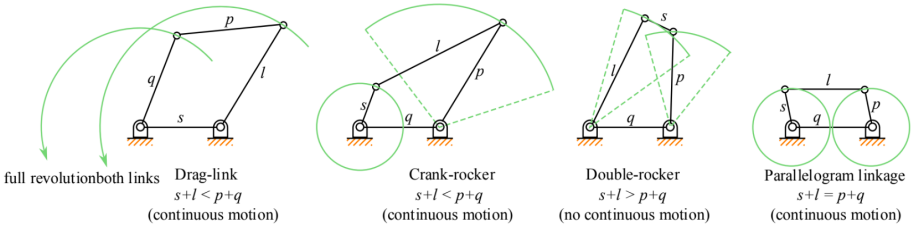


Figura 2.3: Casos del criterio de Grashof [25].

Este criterio constituye una condición preliminar para la síntesis dimensional de un mecanismo.

[3]Es la variante de una cadena cinemática dependiendo del eslabón que se designe como "fijo" o de referencia. Esto no cambia el movimiento relativo entre los eslabones, pero sí el movimiento absoluto generado [24].

2.2.4. Síntesis cinemática

La síntesis de un mecanismo garantiza que, por ejemplo, un manipulador serial alcance los puntos de trabajo con las orientaciones deseadas del efector final [25]. Por esta razón, la síntesis cinemática se suele dividir en pasos: síntesis de tipo o topológica, de número y dimensional. Algunos autores suelen juntar la síntesis de tipo y de número en un solo paso, ya que la primera consiste en determinar el tipo de mecanismo que mejor resuelve el problema planteado y el segundo en determinar la cantidad de eslabones y tipo de juntas para obtener la mobilidad requerida. El último paso está dedicado a definir las dimensiones del mecanismo [25].

De acuerdo con Norton [22], las dos grandes clases que engloban las técnicas existentes de síntesis dimensional son la gráfica o geométrica y la analítica. Los métodos gráficos se basan en principios básicos de geometría euclidiana y se centran en cadenas cinemáticas cerradas de cuatro barras. Varían en función de la cantidad de restricciones de diseño, tales como el número de posiciones, de pivotes móviles y fijos.

Una vez que se encuentre una solución potencial se debe evaluar su calidad. El proceso se vuelve entonces un diseño cualitativo mediante análisis sucesivo [22].

2.2.5. Análisis cinemático

El análisis cinemático permite determinar la configuración, velocidad y aceleración de los elementos móviles de un mecanismo a partir de sus relaciones geométricas. Así se garantiza que no fallará en condiciones de operación [22].

Al considerar el movimiento de un cuerpo rígido se debe partir de la postura o pose, que implica tanto la posición de un punto como la orientación de una línea sobre el eslabón [22], y se expresa con respecto a un marco de referencia[4] asociado a otro eslabón [26]. De esta forma, la pose puede describirse mediante un componente traslacional, dado por el vector de posición ${}^{j}\mathbf{p}_i$, y uno rotacional, dado por la matriz ${}^{j}\mathbf{R}_i$. Con las transformaciones homogéneas, ambos efectos son combinados en una sola representación, como se muestra en la ecuación 2.4 [26]. Además de las matrices de rotación, la orientación puede representarse mediante cuaterniones [26]. En particular, esta representación es utilizada por ROS 2 para describir la orientación tridimensional de los marcos de referencia empleados en sus sistemas de coordenadas [27].

$$ {}^{j}\mathbf{T}_i = \begin{bmatrix} {}^{j}\mathbf{R}_i & {}^{j}\mathbf{p}_i \\ \mathbf{0}^T & 1 \end{bmatrix} \tag{2.4} $$

Para el análisis de posición de un mecanismo plano restringido, éste se representa mediante un lazo vectorial. Los eslabones se representan mediante vectores de posición, cuya suma es igual a cero, como se muestra en la figura 2.4. La notación compleja permite expandir estos vectores mediante la identidad de Euler. De esta forma, se obtiene el sistema de ecuaciones 2.5, con los ángulos θ_3 y θ_4 como incógnitas [22]. Una vez obtenida la configuración del mecanismo, las ecuaciones de velocidad se obtienen

[4] Consiste de un origen O_i y una triada de vectores base $(\hat{x}_i, \hat{y}_i, \hat{z}_i)$.

mediante la diferenciación con respecto al tiempo de las ecuaciones de posición. Una segunda diferenciación permite obtener las ecuaciones de aceleración, utilizando los valores de posición y velocidad correspondientes al instante analizado.

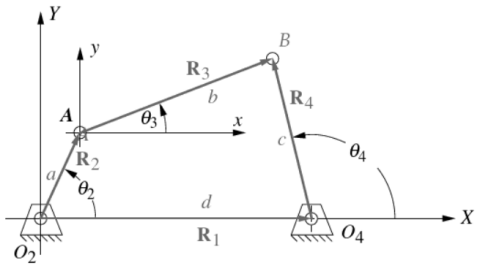


Figura 2.4: Lazo vectorial de posición de un mecanismo de 4 barras [22].

$$\begin{cases} a\cos\theta_2 + b\cos\theta_3 - c\cos\theta_4 - d = 0 \\ a\,\mathrm{sen}\,\theta_2 + b\,\mathrm{sen}\,\theta_3 - c\,\mathrm{sen}\,\theta_4 = 0 \end{cases} \tag{2.5}$$

El mismo procedimiento puede aplicarse al análisis cinemático de un robot serial. Para un manipulador con n grados de libertad, cuando se conocen las variables articulares $\mathbf{q}$ y los parámetros geométricos de los eslabones, es posible resolver el problema de la cinemática directa, mediante el cual se determina la postura del efector final $\mathbf{x}$ con respecto al eslabón base. Esta relación puede expresarse mediante la matriz de transformación homogénea ${}^0\mathbf{T}_n$, resultado de la composición de las transformaciones asociadas a cada junta [26]. Por el contrario, cuando se conoce la pose deseada del efector final, el problema de la cinemática inversa permite determinar las variables articulares correspondientes [26].

A partir de la relación de cinemática directa, su derivada con respecto al tiempo permite establecer la relación entre las velocidades articulares y las velocidades lineal y angular del efector final [26]. Esta relación se expresa mediante el jacobiano, una transformación que relaciona el espacio de juntas con el espacio cartesiano [21], como se muestra en la ecuación 2.6.

$$\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} \tag{2.6}$$

Una segunda diferenciación de la ecuación 2.6 permite obtener la relación entre las aceleraciones articulares y las aceleraciones lineal y angular del efector final, expresado en la ecuación 2.7 [28].

$$\ddot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\ddot{\mathbf{q}} + \dot{\mathbf{J}}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} \tag{2.7}$$

Las relaciones presentadas permiten establecer qué magnitudes deben conocerse durante la operación del sistema. En consecuencia, la selección de los elementos necesarios para su medición constituye el siguiente paso para llevar el modelo cinemático a una implementación física.

2.3. Variables de interés e instrumentación

Llevar el modelo cinemático a una implementación física requiere que el sistema de control conozca el estado del manipulador y el de su entorno durante la operación. Para ello son necesarias tanto una representación interna del sistema, como del entorno en el que se encuentra. Lamentablemente, en la mayoría de los casos ninguna versión lo suficientemente completa o precisa de ambos modelos está disponible. Además, los sistemas de control tampoco son perfectos, por lo que no es realista asumir una respuesta perfecta a la entrada aplicada. Ante esta incertidumbre, el robot debe medir su propio estado y el de su entorno, lo que exige identificar qué variables son relevantes y con qué instrumentos se obtienen.

De las relaciones de cinemática directa (ecs. 2.6 y 2.7) se desprende que las variables de interés de un manipulador son las posiciones, velocidades y aceleraciones articulares ($\mathbf{q}$, $\dot{\mathbf{q}}$, $\ddot{\mathbf{q}}$), a partir de las cuales se obtienen la pose y la velocidad del efector final. A éstas se suman las fuerzas y pares que el manipulador ejerce o recibe, ya que los pares articulares $\boldsymbol{\tau}$ se relacionan con la fuerza y el momento $\mathbf{F}$ en el efector final mediante la transpuesta del jacobiano, como se muestra en la ecuación 2.8 [21, 29, 30].

$$\boldsymbol{\tau} = \mathbf{J}^T(\mathbf{q})\,\mathbf{F} \tag{2.8}$$

Cada una de estas variables se obtiene mediante sensores, dispositivos que convierten una magnitud física en una señal eléctrica procesable. De acuerdo con la magnitud que miden, se clasifican en dos grandes grupos [29–31]:

- *Propioceptivos*: Miden el estado interno del robot, como la posición y velocidad de sus articulaciones, su aceleración o los pares en sus juntas.
- *Exteroceptivos*: Miden el estado del entorno, como la distancia a los obstáculos o la posición de los objetos, a través de cámaras, telémetros láser o sensores de contacto.

Para el control de un manipulador, el sensor propioceptivo más empleado es el codificador (*encoder*) óptico o magnético, que mide la posición angular de una junta [21, 29]. Un codificador incremental entrega N pulsos por revolución, por lo que su resolución angular es $\Delta\theta = 2\pi/N$, mientras que uno absoluto asocia una posición única a cada ángulo y conserva la medición sin necesidad de un punto de referencia al encender el sistema [21, 29]. Por su parte, la velocidad articular puede medirse directamente con un tacómetro o estimarse numéricamente derivando la posición medida [21, 29]. Esta última opción amplifica el ruido de la medición, por lo que suele requerir un filtrado posterior. La aceleración, en cambio, rara vez se obtiene derivando dos veces, y se mide con acelerómetros o unidades de medición inercial (IMU) [31].

La medición de fuerzas y pares se realiza con sensores de fuerza/par ubicados en la muñeca del manipulador, o bien con sensores de par en cada articulación [21, 29, 30]. Estos dispositivos suelen basarse en galgas extensométricas, que transforman la

deformación de un elemento elástico en una señal eléctrica [29]. Esta información es indispensable en tareas donde el robot interactúa con su entorno, como sujetar un objeto sin dañarlo.

Por otro lado, los sensores exteroceptivos permiten ubicar los objetos de interés con respecto a un marco de referencia del robot. Las cámaras RGB-D y los LiDAR de *Justina* entregan nubes de puntos e imágenes, a partir de las cuales se estima la pose de los objetos por manipular [30, 31]. Con ello, la pose objetivo del efector final queda definida como entrada al problema de cinemática inversa.

Sin importar su tipo, ningún sensor entrega una medición exacta. Su salida se describe mediante un modelo de medición $\mathbf{z} = h(\mathbf{x}) + \mathbf{v}$, donde h relaciona el estado real con la magnitud observada y $\mathbf{v}$ representa el ruido [31]. Además de este ruido aleatorio, existen errores sistemáticos, como desfases y factores de escala incorrectos, que se corrigen mediante calibración [31]. Cuando se dispone de varias fuentes de información, como las mediciones de distintos sensores y las predicciones del modelo del sistema, técnicas de estimación como el filtro de Kalman permiten combinarlas para obtener una mejor estimación del estado [31].

Las magnitudes medidas y estimadas constituyen la retroalimentación del sistema. Con ellas, el controlador compara el estado real del manipulador con el deseado y ajusta la señal aplicada a los actuadores, lo cual se aborda en la siguiente sección.

2.4. Control de manipuladores robóticos

La teoría de control proporciona herramientas para diseñar y evaluar algoritmos para realizar los movimientos deseados o las aplicaciones de fuerza. En un manipulador, el problema consiste en determinar los pares que deben aplicar los actuadores para que las variables medidas sigan a las deseadas [21]. Dada una trayectoria articular deseada $\mathbf{q}_d(t)$, el desempeño del sistema se evalúa mediante el error de seguimiento de la ecuación 2.9. Un esquema de lazo abierto no puede corregir las perturbaciones ni las imprecisiones del modelo señaladas en la sección anterior, por lo que se recurre a la retroalimentación de las magnitudes medidas, de modo que el controlador ajuste la acción de control en función del error [21, 32].

$$\mathbf{e} = \mathbf{q}_d - \mathbf{q} \tag{2.9}$$

Para diseñar el controlador es necesario conocer cómo responde el manipulador a los pares aplicados. Esta relación está dada por su modelo dinámico, que para un robot serial de eslabones rígidos se expresa mediante la ecuación 2.10 [21, 32].

$$\boldsymbol{\tau} = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) \tag{2.10}$$

donde:

$\mathbf{M}(\mathbf{q})$ es la matriz de inercia, simétrica y definida positiva.

$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}$ agrupa los términos de fuerzas centrífugas y de Coriolis.

$\mathbf{g}(\mathbf{q})$ es el vector de pares gravitacionales.

Esta ecuación evidencia la principal dificultad del control de manipuladores: la dinámica es no lineal y acopla el movimiento de todas las articulaciones, además de depender de la configuración [32].

El control puede formularse en dos espacios distintos. En el espacio articular, la trayectoria deseada del efector final se convierte previamente a variables articulares mediante la cinemática inversa, y el error se regula directamente sobre $\mathbf{q}$. En el espacio operacional, el error se define sobre la pose del efector final y se transforma al espacio de juntas mediante el jacobiano [21, 32]. La primera opción es más sencilla y se apoya en sensores que miden directamente $\mathbf{q}$. La segunda regula directamente la variable de interés de la tarea, pero exige calcular la cinemática y el jacobiano en cada ciclo de control, y es sensible a las configuraciones singulares, donde éste pierde rango [21].

La estrategia más simple consiste en controlar cada articulación de manera independiente, tratando los efectos de las demás como perturbaciones. Para ello se emplea un controlador proporcional-integral-derivativo (PID) por junta, definido en la ecuación 2.11 [21, 30].

$$\boldsymbol{\tau} = \mathbf{K}_p\,\mathbf{e} + \mathbf{K}_d\,\dot{\mathbf{e}} + \mathbf{K}_i \int_0^t \mathbf{e}\,d\tau \tag{2.11}$$

donde $\mathbf{K}_p$, $\mathbf{K}_d$ y $\mathbf{K}_i$ son matrices diagonales de ganancias proporcional, derivativa e integral, respectivamente. Cuando la articulación se modela como un sistema de segundo orden con inercia efectiva I_{ef}, la respuesta es críticamente amortiguada si $k_d = 2\sqrt{k_p I_{ef}}$ [21]. La acción integral elimina el error en estado estable producido por perturbaciones constantes, como la gravedad, aunque puede deteriorar la respuesta transitoria [21]. Este esquema funciona razonablemente cuando las relaciones de reducción son altas, ya que éstas atenúan el acoplamiento y la variación de la inercia. Sin embargo, el desempeño se degrada ante movimientos rápidos o cargas variables, porque ignora el modelo de la ecuación 2.10. Una mejora sencilla es sumar la compensación de gravedad $\mathbf{g}(\mathbf{q})$ a un control proporcional-derivativo, con lo que se logra llevar el manipulador a una posición deseada constante [30, 32].

Para el seguimiento de trayectorias se emplea el control por dinámica inversa o de par calculado. Esta estrategia utiliza el modelo del manipulador para cancelar los términos no lineales y desacoplar las articulaciones, como se muestra en la ecuación 2.12 [21, 32].

$$\boldsymbol{\tau} = \mathbf{M}(\mathbf{q})\left(\ddot{\mathbf{q}}_d + \mathbf{K}_d\dot{\mathbf{e}} + \mathbf{K}_p\mathbf{e}\right) + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) \tag{2.12}$$

Si el modelo es exacto, al sustituir la ecuación 2.12 en la ecuación 2.10 se obtiene $\ddot{\mathbf{e}} + \mathbf{K}_d\dot{\mathbf{e}} + \mathbf{K}_p\mathbf{e} = \mathbf{0}$, un sistema lineal cuyo comportamiento se define eligiendo las ganancias [21, 32]. Esta ley requiere medir $\mathbf{q}$ y $\dot{\mathbf{q}}$, y depende de la fidelidad del modelo, que en la práctica contiene errores en las masas, inercias y fricción. Para reducir esta sensibilidad existen estrategias de control robusto, adaptable y de aprendizaje [29, 32].

Las estrategias anteriores controlan el movimiento del manipulador, lo cual es adecuado mientras éste se desplaza libremente. Cuando el efector final entra en contacto con el entorno, como al tomar o dejar un objeto, el control puramente de posición resulta inadecuado, pues pequeños errores de posición producen grandes fuerzas de contacto, sobre todo en entornos rígidos [21, 33]. En estos casos es necesario controlar la interacción mediante la retroalimentación de los sensores de fuerza descritos en la sección anterior. Las estrategias principales se agrupan en [33]:

- *Control indirecto*: Regula la relación dinámica entre la desviación de la pose del efector final y la fuerza de contacto, mediante control de rigidez o de impedancia, de modo que el manipulador se comporte de forma compliante sin requerir un modelo preciso del entorno [34].
- *Control híbrido de posición/fuerza*: Divide los grados de libertad de la tarea según sus restricciones, de forma que se controla la posición en las direcciones libres y la fuerza en las direcciones restringidas por el entorno [21, 35].

Capítulo 3

Metodología

Históricamente, el conocimiento práctico y la experiencia acumulada constituyeron la base del diseño. Como señalan Pahl y Beitz, a partir de la segunda mitad del siglo XX, la importancia económica del desarrollo eficiente y oportuno de productos motivó la creación de procedimientos sistemáticos y flexibles que permitieran planificar, optimizar, verificar y enseñar el proceso de diseño. Tal evolución propició la formalización de metodologías para analizar y descomponer problemas, generar soluciones óptimas y evaluar alternativas bajo criterios técnicos y económicos [36]. Estas metodologías, sin embargo, partían de especificaciones y requisitos que no consideraban a quienes usarían los productos.

El diseño centrado en el usuario (DCU) surgió en el laboratorio de Donald A. Norman en la Universidad de California San Diego como una filosofía que involucra al usuario en todas las fases del desarrollo de un producto, desde su conceptualización hasta su evaluación [37]. Norman propuso en *The Design of Everyday Things* un enfoque que parte de la comprensión profunda de las personas y de las necesidades que el diseño pretende satisfacer [38]. Este enfoque no es subjetivo, pues se fundamenta en disciplinas científicas como la ergonomía, la psicología y la antropología, y requiere que las decisiones de diseño estén respaldadas por datos obtenidos directamente de los usuarios [39].

En aplicaciones domésticas de robótica de servicio, el DCU resulta especialmente pertinente. Robots como *Justina* demuestran capacidades de navegación, manipulación, visión e interacción humano-robot que les permiten operar en ambientes reales no estandarizados, adaptándose a los hogares y a sus habitantes [40]. Identificar sistemáticamente las necesidades de quienes conviven con estos sistemas es, por lo tanto, una condición necesaria para orientar el desarrollo de soluciones robóticas apropiadas.

Bajo esta perspectiva, el rediseño de los manipuladores de *Justina* adoptó la metodología de Karl T. Ulrich y Steven D. Eppinger, quienes presentan en *Diseño y Desarrollo de Productos* un conjunto de métodos para el diseño de productos físicos, discretos e ingenieriles [41]. Su proceso genérico se divide en seis fases: planeación, desarrollo del concepto, diseño a nivel sistema, diseño de detalle, pruebas y refinamiento, e inicio de producción.

Este capítulo abarca las fases de desarrollo del concepto y diseño a nivel sistema, que comprenden la identificación de necesidades, el establecimiento de especificaciones, la descomposición funcional, la generación de conceptos y la selección del concepto que se llevará al diseño de detalle.

3.1. Identificación de necesidades

La identificación de necesidades del cliente es la fase fundacional del proceso. Su objetivo es capturar de manera sistemática y exhaustiva las necesidades, expectativas y restricciones de los usuarios potenciales, transformando información cualitativa en un conjunto estructurado de requerimientos. Ulrich y Eppinger [41] definen una necesidad del cliente como una descripción, en las propias palabras del usuario, de los beneficios que un producto debe proporcionar, y distinguen con claridad entre necesidades —que describen *qué* debe lograr el producto— y soluciones —que especifican *cómo* lograrlo— . El proceso se estructura en cinco pasos: recopilación de datos sin procesar mediante entrevistas, interpretación de esos datos en términos de necesidades, organización jerárquica, establecimiento de importancia relativa y reflexión sobre los resultados.

3.1.1. Perfiles de usuario

Para la implementación del proceso de identificación de necesidades se definieron dos perfiles de participantes mediante muestreo intencional (Apéndice B).

El primero es el **usuario final**: personas que requieren asistencia doméstica constante debido a edad avanzada o alguna condición que limite su autonomía en actividades cotidianas del hogar, con capacidad cognitiva para participar en una entrevista y proporcionar consentimiento informado (Apéndice B). Sus necesidades se relacionan con la utilidad práctica del robot, la facilidad de interacción, la confiabilidad y la seguridad durante el uso.

El segundo perfil corresponde a **desarrolladores de software con experiencia en robótica de servicio**: miembros activos o previos del equipo de desarrollo de Justina con experiencia de participación en RoboCup@HOME y conocimiento técnico sobre el diseño, programación o implementación de funcionalidades del robot (Apéndice B). Este perfil aporta una perspectiva complementaria: han trabajado directamente con robots de servicio en condiciones desafiantes, han observado y comparado múltiples plataformas en competencia, y constituyen también un tipo de usuario del sistema al ser quienes realizan el mantenimiento y la programación del robot.

Dentro de este segundo perfil se distinguió entre quienes trabajan con **plataformas experimentales** de diseño propio y quienes utilizan **plataformas comerciales**, específicamente el *Toyota Human Support Robot* (Takeshi) del Laboratorio de Bio-Robótica (Apéndice B). Esta distinción motivó el diseño de variantes específicas en los instrumentos de recolección.

3.1.2. Recopilación de datos

El proyecto contó con la aprobación del Comité de Ética en Investigación y Docencia de la Facultad de Ingeniería de la UNAM (Apéndice C). Todos los participantes firmaron una carta de consentimiento informado antes de su participación, bajo la cual se garantizó la confidencialidad de la información mediante codificación de datos, almacenamiento seguro y acceso restringido al equipo investigador (Apéndice C).

Siguiendo las recomendaciones de Ulrich y Eppinger [41], se seleccionaron las entrevistas como método principal de recopilación. Se diseñaron cuatro guiones semiestructurados diferenciados según perfil: uno para el usuario final, uno de aplicación general para desarrolladores, y dos variantes específicas para desarrolladores de plataformas experimentales y comerciales, respectivamente (Apéndice B). Los guiones completos se documentan en el Anexo [X].

Se realizaron nueve entrevistas en total: cinco con desarrolladores con experiencia en RoboCup@HOME y cuatro con usuarios finales potenciales. Todas fueron grabadas en audio con consentimiento explícito de los participantes (Apéndice C). Los fragmentos relevantes se transcribieron y organizaron en tablas estructuradas que vinculan cada declaración con su participante, el contexto de la pregunta y anotaciones sobre necesidades implícitas.

3.1.3. Interpretación y organización jerárquica

Siguiendo las directrices de Ulrich y Eppinger [41], cada necesidad se expresó como un atributo del producto —sujeto, verbo que describe la acción o capacidad deseada y, cuando es necesario, objeto o contexto de aplicación— capturando el *qué* sin anticipar el *cómo*. Se adoptó un enfoque holístico que abarcó el robot de servicio doméstico en su totalidad, reconociendo que las necesidades de manipulación no existen de manera aislada sino interrelacionadas con navegación, percepción e interacción. Como resultado, se identificaron **122 necesidades distintas**, mencionadas colectivamente **690 veces** a lo largo de todas las entrevistas.

Las 122 necesidades se organizaron en una jerarquía de **14 necesidades primarias** con sus correspondientes necesidades secundarias y terciarias. El agrupamiento se realizó en una hoja de cálculo de Excel, lo que permitió reorganizar iterativamente los enunciados y mantener trazabilidad entre cada necesidad y su fuente original. La estructura jerárquica completa se documenta en el Anexo [Y].

3.1.4. Importancia relativa y selección

Para determinar la importancia relativa de las 14 necesidades primarias se implementó una estrategia combinada. El primer componente fue la **frecuencia relativa de mención**: el cociente entre el número de menciones de cada necesidad primaria y sus derivadas sobre el total de 690 menciones. El segundo componente fue una **encuesta estructurada** distribuida electrónicamente a tres de los participantes entrevistados,

cuyos resultados se interpretaron como indicadores cualitativos de tendencia complementarios a los datos de frecuencia, dado el tamaño reducido de la muestra. La escala utilizada constó de cinco niveles: *Imprescindible* (1.0), *Deseable* (0.6), *Indiferente* (0.4), *Preferiría que no la tuviera* (0.2) y *No debería tenerla* (0.0). La importancia relativa final se calculó combinando el promedio de las evaluaciones directas con la frecuencia de mención de cada necesidad.

Aplicando un umbral de corte basado en la distribución de valores obtenidos, se seleccionaron seis necesidades primarias prioritarias. De estas, se retuvieron tres con implicación directa en el diseño de los manipuladores: *el robot manipula artículos domésticos*, *el robot es robusto* y *el robot tiene un diseño intuitivo*. A la primera se le incorporaron cuatro necesidades secundarias que detallan aspectos específicos de la capacidad de manipulación. El conjunto final quedó conformado por:

1. **El robot manipula artículos domésticos.**
   - El robot planea trayectorias del manipulador.
   - El robot toma artículos con la fuerza necesaria.
   - El robot extiende el alcance efectivo de sus manipuladores.
   - El robot manipula equipo para realizar tareas domésticas.
2. **El robot es robusto.**
3. **El robot tiene un diseño intuitivo.**

3.2. Establecimiento de especificaciones objetivo

La segunda fase de la metodología consiste en traducir las necesidades del cliente en especificaciones técnicas precisas y medibles. Ulrich y Eppinger [41] distinguen entre necesidades —expresadas en el lenguaje subjetivo del usuario— y especificaciones —que describen con exactitud cuantificable qué debe ser el producto—. Una especificación consiste en una *métrica* y un *valor* acompañado de sus unidades. Las especificaciones se establecen en dos momentos: primero como **especificaciones objetivo**, antes de conocer las restricciones técnicas del concepto de diseño, y luego como **especificaciones finales**, una vez seleccionado el concepto. El proceso comprende cuatro pasos: elaborar la lista de métricas, recabar información de referencia mediante *benchmarking*, establecer valores meta ideales y marginalmente aceptables, y reflexionar sobre los resultados.

3.2.1. Traducción de necesidades a métricas

Para cada necesidad del cliente se identificó la característica precisa y medible del producto que mejor refleja el grado en que esa necesidad queda satisfecha. La relación entre necesidades y métricas se documentó en una matriz de correlación (Cuadro 3.2) que evidencia qué métricas cubren cada necesidad y permite identificar aquellas que

requieren múltiples indicadores. A partir de las necesidades identificadas se definieron las ocho métricas presentadas en el Cuadro 3.1.

Cuadro 3.1: Lista de métricas derivadas de las necesidades del cliente.

#	**Métrica**	**Nec. relacionadas**	**Importancia**	**Unidad**
1	Capacidad de carga	1, 5, 6	5	[kg]
2	Fuerza de agarre	1, 3, 5	5	[N]
3	Alcance	2, 4, 5	5	[mm]
4	Apertura	1, 5	4	[mm]
5	Grados de libertad	4, 5	4	[1]
6	Repetibilidad	2, 5, 6	4	[mm]
7	Grado de protección IP	5, 6	3	[1]
8	Mapa de desensamble	6, 7	3	[s]

Cuadro 3.2: Matriz de correlación entre necesidades del cliente y métricas.

Necesidad	**M1**	**M2**	**M3**	**M4**	**M5**	**M6**	**M7**	**M8**
El robot manipula artículos domésticos	×	×		×				
El robot planea trayectorias del manipulador			×			×		
El robot toma artículos con la fuerza nec.		×						
El robot extiende el alcance efectivo			×		×			
El robot manipula equipo para tareas dom.	×	×	×	×	×	×	×	
El robot es robusto	×					×	×	×
El robot tiene un diseño intuitivo								×

3.2.2. Benchmarking y análisis competitivo

Para establecer valores de referencia se realizó un análisis comparativo de sistemas representativos de dos categorías: *cobots* de uso industrial —Panda/Franka Emika, Gen3/Kinova, UR3e/Universal Robots, EBLM/Ti5 Robot y OpenArm— y manipuladores de robots de servicio —Stretch 3/Hello Robot, HSR/Toyota, TIAGo/PAL Robotics y G1/Unitree—. Para los efectores finales se compararon grippers de dos dedos (Franka Hand, 2F-85 y 2F-140/Robotiq, HSR) y de tres dedos (3F-155/Robotiq, 3F-AR/UW-Milwaukee, DG-3F-B/Tesollo, Dex3s/Unitree).

La evaluación se realizó mediante un análisis de calidad de función (QFD) que ponderó cada métrica según su importancia relativa a las necesidades primarias identificadas. Para el brazo se evaluaron cuatro grupos de necesidades: planeación de trayectorias (alcance y repetibilidad), alcance efectivo (alcance y grados de libertad), tareas domésticas (capacidad de carga, alcance, grados de libertad, repetibilidad e IP) y robustez (capacidad de carga, repetibilidad e IP). Para el gripper se evaluaron las necesidades de manipulación (capacidad de carga, fuerza de agarre, apertura, grados de libertad y repetibilidad), control de fuerza, tareas domésticas y robustez.

3.2.3. Definición de valores objetivo

Con base en el análisis de referencia se establecieron valores objetivo para cada métrica en dos niveles: el **valor ideal**, que representa el mejor desempeño razonablemente alcanzable, y el **valor marginal**, que constituye el mínimo aceptable para que el diseño sea viable. El Cuadro 3.3 presenta el conjunto completo de especificaciones objetivo.

Cuadro 3.3: Especificaciones objetivo para los manipuladores de Justina.

#	**Métrica**	**Imp.**	**Unidad**	**V. marginal**	**V. ideal**
1	Capacidad de carga	5	[kg]	> 2	> 3
2	Fuerza de agarre	5	[N]	> 70	> 100
3	Alcance	5	[mm]	> 500	> 700
4	Apertura	4	[mm]	> 100	> 140
5	Grados de libertad	4	[1]	≥ 5	≥ 6
6	Repetibilidad	4	[mm]	$< \pm 1$	$< \pm 0{,}1$
7	Grado de protección IP	3	[1]	IP44	IP55
8	Mapa de desensamble	3	[s]	—	—

La métrica M8 (mapa de desensamble), correspondiente a la necesidad de diseño intuitivo, se retiene en el conjunto como requerimiento cualitativo: su naturaleza no admite una cuantificación directa en esta etapa y su evaluación se formalizará durante la fase de selección de conceptos.

3.3. Generación de conceptos

La generación de conceptos es la actividad mediante la cual el equipo de desarrollo explora de manera sistemática el espacio de alternativas de diseño posibles antes de comprometerse con una solución particular. Ulrich y Eppinger [41] definen el concepto de un producto como una descripción aproximada de la tecnología, los principios de funcionamiento y la forma del producto. El método estructurado propuesto consta de cinco pasos: aclarar el problema mediante descomposición funcional, buscar externamente conceptos existentes, generar internamente nuevas soluciones, explorar sistemáticamente las combinaciones posibles y reflexionar sobre los resultados. Dado que la generación de conceptos es relativamente económica en comparación con las fases posteriores del desarrollo, los autores argumentan que no existe justificación para limitarla, y que un equipo eficiente debería explorar el espacio de alternativas con amplitud suficiente como para tener confianza en que no ha pasado por alto soluciones superiores.

3.3.1. Descomposición funcional del problema

El primer paso consistió en representar el sistema de manipulación como una caja negra que opera sobre flujos de energía, material y señales, para luego dividirla en

subfunciones que describen lo que los elementos del manipulador deben hacer. De la totalidad de subfunciones identificadas se seleccionaron los **seis subproblemas críticos** que determinan en mayor medida el desempeño del sistema: aceptar energía, convertir energía a movimiento, transmitir movimiento y fuerza, sensar posición, sensar fuerza y sostener el objeto. El diagrama funcional completo, que muestra la interacción entre estas subfunciones a través de los flujos de energía y señal, se presenta en la Figura

3.3.2. Búsqueda externa de soluciones

Para cada uno de los seis subproblemas críticos se realizó una búsqueda exhaustiva de soluciones existentes en la literatura técnica, catálogos de fabricantes, patentes y en el análisis directo de los robots estudiados durante el benchmarking. Las soluciones encontradas se organizaron en tablas morfológicas —una por subproblema— que cubren los principales principios de solución para cada función. Las tablas completas se incluyen en el Anexo [W].

3.3.3. Exploración sistemática y acotamiento del espacio de soluciones

La combinación irrestricta de las soluciones identificadas genera un espacio de diseño de gran magnitud. Mediante un script de Python se enumeraron sistemáticamente **1943 combinaciones** que representan el espacio de búsqueda completo antes de aplicar criterios de compatibilidad.

Para reducir este espacio a candidatos viables se procedió en dos etapas. Primero, se acotó el conjunto de opciones por subproblema a las soluciones más pertinentes para el contexto de aplicación: robot de servicio doméstico, operación eléctrica e integración con el stack de software existente del laboratorio. Segundo, sobre este espacio reducido se aplicó un criterio de **afinidad entre soluciones**: se conservaron únicamente aquellas combinaciones que representan configuraciones lógicamente coherentes y directamente implementables, que no requieren adaptaciones innecesarias para funcionar conjuntamente. Este filtrado redujo el espacio de candidatos a **384 combinaciones realistas**.

3.3.4. Desarrollo de conceptos completos

Del conjunto de 384 candidatos se seleccionaron **4 conceptos representativos** para su desarrollo en detalle, buscando cubrir la diversidad de enfoques presentes en el espacio de soluciones. Cada concepto se materializó en un modelo CAD conceptual que permitió evaluar de manera preliminar las métricas establecidas en las especificaciones objetivo y facilitar la comparación estructurada en la fase de selección. Los cuatro conceptos, sus características principales y su evaluación frente a las especificaciones se describen en el capítulo siguiente.

Apéndice A

Justina

La arquitectura de software de *Justina*, robot de servicio doméstico del Laboratorio de Bio-Robótica de la UNAM, se implementa sobre ROS 2 y distribuye sus paquetes en dos categorías de módulos:

- **Módulos orquestadores**: Administran la operación autónoma (*Planning*, *Supervisor* y *Executor*).
- **Módulos especializados**: Proporcionan capacidades específicas (*Knowledge*, *HRI*, *Vision*, *Navigation*, *Manipulation* y *Hardware*).

La estructura de cada módulo, especificando sus paquetes, nodos e interfaces de comunicación, se documenta a continuación.

A.1. Planning

El módulo *Planning* traduce comandos en lenguaje natural o metas estructuradas en un plan ejecutable de instrucciones. El procesamiento inicia en las herramientas de análisis semántico (*semantic_parser*) y concluye en el motor de reglas CLIPS (`clips_node`), el cual publica la secuencia de acciones resultante en `/planning/clips/results` para su posterior gestión por el módulo *Supervisor*.

A.1.1. Paquete `clips_node` (Python)

Proporciona la envoltura para el motor de reglas CLIPS (mediante la librería `clips`) y las utilidades de conversión de metas.

1. **clips_node**:
   Motor principal de planeación. Carga el conjunto de reglas definido por el parámetro `plan_name` (correspondiente a categorías de prueba de RoboCup@Home como `gpsr_planning`, `restaurant_planning`, entre otros) y ejecuta la inferencia en un hilo independiente para evitar el bloqueo del ejecutor de ROS 2.

- **Tópicos publicados**:
  `/planning/clips/results` (`std_msgs/String`).
- **Servicios ofrecidos**:
  `/planning/clips/start_planning`,
  `/planning/clips/assert_fact`,
  `/planning/clips/assert_facts_batch`.

2. **goals_to_clips_node**:
   (Nombre interno: `goals_to_clips`) Convierte una lista JSON de metas `goal(args)` en hechos CLIPS, realizándolo en lote antes de disparar la planeación.

   - **Servicios ofrecidos**:
     `/planning/goals_to_clips/convert`.
   - **Servicios utilizados**:
     `/planning/clips/assert_facts_batch`,
     `/planning/clips/start_planning`.

3. **natural_to_clips.py**:
   Estado: Ejecutable instalado pero huérfano (sin referencias en archivos *launch*). Traduciría oraciones libres a hechos CLIPS.

   - **Servicios ofrecidos**:
     `/hri/listener/natural_to_clips/interpret`.

A.1.2. Paquete `semantic_parser`

Ofrece capacidades de comprensión de lenguaje natural para la interpretación de comandos.

1. **qwen_semantic_node**:
   Consulta un servidor HTTP externo basado en el modelo Qwen para transformar comandos transcritos en metas estructuradas.

   - **Servicios ofrecidos**:
     `/planning/semantic_parser/semantic_parser`,
     `/planning/semantic_parser/listen_gpsr`.
   - **Servicios utilizados**:
     `/hri/listener/listen`.

2. **semantic_node**:
   Analiza oraciones mediante un *pipeline* de spaCy (`en_core_web_trf`) para generar representaciones semánticas estructuradas.
   Anomalía de despliegue: El archivo `semantic_parser.launch.xml` instancia este nodo dos veces de forma simultánea. Esto genera una colisión de nombres de nodo e interfiere con el servicio registrado por `qwen_semantic_node`.

- **Servicios ofrecidos**:
  `/planning/semantic_parser/semantic_parser_spacy`,
  `/planning/semantic_parser/semantic_parser`.

3. **entity_extractor_node**:
   Identifica y extrae entidades nombradas consultando directamente los diccionarios del paquete `knowledge_bank`.

   - **Servicios ofrecidos**:
     `/planning/semantic_parser/extract_entity`.

A.1.3. Paquete `primitive_to_clips`

Convierte la descripción de una primitiva en hechos CLIPS contextualizados con el estado del entorno.

1. **primitive_to_clips_node**:
   Estado: Inactivo (comentado en `clips.launch.xml`).

   - **Servicios ofrecidos**:
     `/planning/primitive_to_clips/convert`.
   - **Servicios utilizados**:
     `/planning/clips/assert_facts_batch`,
     `/planning/clips/start_planning`.

A.1.4. Paquete `planning_tools` (C++)

Funciona como la fachada principal (clase `Planning`) y aloja el archivo *launch* raíz del módulo (`planning.launch.xml`). No contiene nodos ejecutables persistentes propios, pero agrupa las llamadas del cliente a los servicios de todo el módulo.

Anomalías detectadas: Los métodos `interpretToClips()` y `requestInstruction()` hacen referencia a servicios cuyos servidores no están activos en el despliegue actual (`natural_to_clips.py` inactivo y discrepancia en el *namespace* de `supervisor_plan_node`).

A.1.5. Paquetes de soporte y código de depuración

- `clips_launch`:
  Paquete exclusivo de archivos *launch* (`clips.launch.xml`) que orquesta el arranque de `clips_node` y `goals_to_clips_node`.

- `plan_manager_node` (**Código huérfano**):
  Nodo no referenciado por ningún archivo *launch*. Implementa lógica de despacho previa que ha sido reemplazada por el nodo `supervisor_plan_node` en el módulo *Supervisor*.

A.2. Supervisor

El módulo *Supervisor* cierra el ciclo entre *Planning* y *Executor*: convierte el plan serializado que produce CLIPS en una cola de instrucciones que el *Executor* solicita secuencialmente, valida y clasifica el resultado de cada tarea reportada y (cuando está habilitado) orquesta comportamientos correctivos entre ambos.

> **Hallazgo crítico de despliegue**: En la configuración activa (`supervisor.launch.xml`), el nodo coordinador principal (`supervisor_manager_node`) se encuentra comentado y **no se lanza**. Al estar desactivado, no existe ningún nodo en el sistema que publique la señal `/supervisor/supervisor_manager/allow_next_instruction`, omitiendo el pulso automático hacia `/input_trigger` necesario para avanzar secuencialmente entre instrucciones.

A.2.1. Paquete `supervisor_plan`

Recibe el plan serializado publicado por CLIPS (`/planning/clips/results`), lo parsea en una cola interna de instrucciones secuenciales y las expone bajo demanda. Soporta la superposición de instrucciones correctivas sin perder la cola original y permite rebobinar pasos. Si no hay un plan cargado, entrega una instrucción de respaldo (`"Justina listen keyword"`).

1. **supervisor_plan_node**:
   Mantiene la cola de instrucciones del plan activo y gestiona el despacho bajo demanda.

   - **Tópicos publicados**:
     `/instruction_msg` (`std_msgs/String`),
     `/supervisor/supervisor_plan/active` (`std_msgs/Bool`),
     `/supervisor/supervisor_plan/plan_ready` (`std_msgs/Empty`).
   - **Tópicos suscritos**:
     `/planning/clips/results`,
     `/supervisor/supervisor_manager/instruction`,
     `/supervisor/supervisor_manager/instruction_command`,
     `/supervisor/supervisor_manager/allow_plan_instruction`,
     `/supervisor/supervisor_plan/rewind_one`.
   - **Servicios ofrecidos**:
     `/request_instruction`.

A.2.2. Paquete `supervisor_execution`

Valida y clasifica el resultado de cada tarea reportada por `executor_manager` tras la ejecución en `executor_worker`. Contiene una clase validadora por cada acción del

Executor (`GoTo`, `Say`, `Listen`, `Take`, `Drop`, entre otras), la cual normaliza el resultado crudo devuelto por las herramientas a un reporte estructurado (`TaskExecutionReport`).

1. **supervisor_execution_node**:
   Autoriza y clasifica la ejecución de tareas descolgando hacia el validador de la acción correspondiente y emite el pulso de disparo hacia el Executor.

   - **Tópicos publicados**:
     `/input_trigger` (`std_msgs/Empty`),
     `/supervisor/supervisor_execution/result` (`TaskExecutionReport`).
   - **Tópicos suscritos**:
     `/supervisor/supervisor_manager/allow_next_instruction`.
   - **Servicios ofrecidos**:
     `/supervisor/supervisor_executor/authorize`.

A.2.3. Paquete `supervisor_manager`

Coordinador de alto nivel del módulo. Incorpora la política de recuperación PNP (`PnpRecoveryManager`) para pruebas de *Pick and Place* (reintentos locales, verificación de agarre con `verify_take`, posposición de objetos fallidos y reordenamiento de tareas) y mantiene clientes directos hacia los módulos de HRI, Navegación y Hardware para ejecutar acciones correctivas independientes del Executor.

1. **supervisor_manager_node**:
   Estado: Inactivo (comentado en `supervisor.launch.xml`).

   - **Tópicos publicados**:
     `/supervisor/supervisor_plan/rewind_one`,
     `/supervisor/supervisor_manager/allow_next_instruction`,
     `/supervisor/supervisor_manager/allow_plan_instruction`,
     `/supervisor/supervisor_manager/instruction_command`,
     `/supervisor/supervisor_manager/instruction`.
   - **Tópicos suscritos**:
     `/system/status`,
     `/supervisor/supervisor_execution/result`,
     `/supervisor/supervisor_plan/active`,
     `/supervisor/supervisor_plan/plan_ready`,
     `/instruction_msg`.
   - **Servicios utilizados**:
     `/hri/talker/talk`,
     `/hri/listener/listen`,
     `/nav_controller/go_to_location`,
     `/hardware/head/move_head`.

A.2.4. Paquete `supervisor_system`

Monitorea la salud global del sistema emitiendo el estado operativo de los subsistemas del robot.

1. **supervisor_system_node**:
   (Nombre interno del nodo: `system_status_publisher`)
   Limitación de implementación: El nodo no inspecciona activamente el grafo de ROS 2 ni verifica procesos vivos; publica de forma fija (*hardcoded*) un estado saludable para todos los módulos cada segundo.

   - **Tópicos publicados**:
     `/system/status`.

A.2.5. Paquete `surge_et_ambula`

Paquete de orquestación (sólo archivos *launch*) que centraliza el arranque global del robot. Contiene los lanzadores raíz para el robot real (`justina.launch.xml`), el entorno de simulación (`justina_sim.launch.xml`), mapas (`map.launch.xml`) y configuraciones alternativas de hardware.

A.3. Executor

El módulo *Executor* ejecuta secuencialmente las instrucciones del plan entregado por *Planning/Supervisor*. Se encarga de solicitar cada instrucción, traducirla a una meta de acción concreta y despachar su ejecución mediante máquinas de estados finitos (FSM) que orquestan las capacidades del robot (navegación, manipulación, HRI, visión y hardware), reportando el resultado de vuelta al Supervisor.

A.3.1. Paquete `executor_manager`

Funciona como puente entre el *Supervisor* y la capa de ejecución física. Solicita la instrucción, convierte el texto crudo (p. ej. *"Justina goto table"*) en una meta estructurada de acción, la despacha y gestiona el ciclo de vida de la tarea (transición al siguiente paso, salto por rechazo o reintento configurable hasta `MAX_RETRIES`).

1. **executor_manager_node**:
   Orquesta el flujo de ejecución del plan y reacciona a los estados devueltos por el servidor de acciones (`SUCCEEDED`, `REJECTED`, `FAILURE`, `ABORTED`).

   - **Tópicos suscritos**:
     `/input_trigger` (`std_msgs/Empty`).

- **Servicios utilizados**:
  `/request_instruction`,
  `/supervisor/supervisor_executor/authorize`.
- **Acciones utilizadas**:
  `/executor/execute_task` (`executor_interfaces/action/ExecuteTask`).

A.3.2. Paquete `executor_worker`

Capa de ejecución física que expone el servidor de acciones de ROS 2. Sigue una política de *"accept-to-report"* (acepta incluso metas inválidas para notificar formalmente el rechazo al Supervisor) y despacha cada tarea aceptada en un hilo independiente para evitar el bloqueo del nodo.

1. **executor_worker_node**:
   Mapea el campo de acción recibido hacia la clase FSM correspondiente en `executor_tools`.

   - **Acciones ofrecidas**:
     `/executor/execute_task` (`executor_interfaces/action/ExecuteTask`).

A.3.3. Paquete `executor_tools` (Librería FSM)

Librería C++ de máquinas de estado finitos que encapsulan las primitivas de acción del robot. Cada FSM se comunica con las librerías `*_tools` de otros módulos y devuelve un objeto `Result` uniforme (`success`, `state`, `message`).

Las primitivas registradas en el despachador (`actionMap`) son:

- `say` (`Say`): Pronuncia una frase del banco de conocimiento mediante sintetizador de voz (TTS).
- `listen` (`Listen`): Captura audio, extrae entidades nombradas y actualiza el estado del mundo.
- `go_to` (`GoTo`): Ajusta la altura del torso y navega a una ubicación nombrada.
- `check` (`Check`): Verifica el estado de un componente de hardware o sensor (p. ej. estado de una puerta).
- `point` (`Point`): Alinea la base y la cabeza hacia un objetivo y extiende el brazo para señalarlo.
- `give` (`Give`): Extiende el brazo que sostiene un objeto y abre el *gripper* tras confirmar la recepción.
- `receive` (`Receive`): Extiende el brazo con el *gripper* abierto y lo cierra al detectar el objeto entregado por una persona.

- `take` (`Take`): FSM principal de agarre. Ejecuta la aproximación visual fina mediante la clase de soporte `VisualGraspApproach` (cálculo de candidatos `FRONT`/`TOP` y corrección por TF del marcador), planea con `manipulation_tools`, cierra el *gripper* y retrae la trayectoria.
- `verify_take` (`VerifyTake`): Confirma por visión la presencia del objeto tras la maniobra de agarre.
- `drop` (`Drop`): Consume una TF de espacio seguro producida por `FindSpace` y deposita el objeto.
- `find_space` (`FindSpace`): Localiza y valida una TF de espacio libre sobre superficies (mesas, repisas) para depositar objetos.
- `find_object` (`FindObject`): Escanea con la cabeza, detecta objetos por visión y actualiza su pose en el mundo.
- `find_person` (`FindPerson`): Detecta, localiza e identifica a una persona, actualizando su registro en el estado del mundo.
- `find_pose` (`FindPose`): Detecta la postura de una persona y la almacena en un búfer compartido.
- `approach` / `approach_partial` (`Approach`): Aproxima al robot a una persona u objeto. La variante `approach_partial` invoca `runPartial` para acercamientos controlados e incompletos.
- `align` (`Align`): Alinea la base respecto a un objetivo visual mediante control en lazo cerrado.
- `focus` (`Focus`): Centra la cámara de la cabeza en un objetivo previamente detectado.
- `follow` (`Follow`): Habilita o deshabilita el servicio de seguimiento de personas.
- `analyze_objects` (`AnalyzeObjects`): Realiza análisis de conteo, tamaño o color en objetos detectados.
- `analyze_person` (`AnalyzePerson`): Analiza vestimenta o gestos de una persona.
- `save_loc` (`SaveLoc`): Guarda la pose actual o relativa del robot como una nueva ubicación nombrada.
- `fold` (`Fold`): Dobla una prenda sobre una mesa en dos fases (mangas y base) guiándose por puntos clave de visión.

A.3.4. Paquete `gui_controller` (Herramienta de desarrollo)

Aplicación de escritorio desarrollada en PyQt5 para monitoreo y depuración durante el desarrollo. Permite lanzar, supervisar y detener procesos en el sistema.

1. **gui_controller** (`ROS2NodeChecker`):
   Nodo auxiliar que inspecciona el grafo del sistema mediante las APIs de `ros2node` y permite finalizar procesos mediante señales del sistema (`pkill`). No forma parte del grafo de ejecución en tiempo de competencia ni expone tópicos o servicios propios.

A.4. Knowledge

El módulo *Knowledge* se encarga de mantener y exponer el estado del mundo del robot en tiempo de ejecución (personas, objetos, ubicaciones conocidas, atributos del robot y analítica de percepción). Asimismo, provee un banco de frases parametrizables que el módulo HRI utiliza para la generación de diálogo, resolviendo dinámicamente las referencias de dichas frases contra el estado del mundo actualizado.

A.4.1. Paquete `knowledge_bank`

Paquete de datos (sin nodos) que centraliza la configuración estática y el estado persistente. Contiene:

- **Dominios estáticos**: `domains/locations.yaml`, `objects.yaml` y `people.yaml`.
- **Banco de frases**: Organizado por contextos de pruebas en `phrases/` (`gpsr.yaml`, `hri.yaml`, `inspection.yaml`, `laundry.yaml`, `pnp.yaml`, `restaurant.yaml`).
- **Vocabulario**: Diccionarios en `dictionaries/` (bebidas, nombres, objetos, órdenes).
- **Estado en tiempo de ejecución**: `runtime/world.yaml`, archivo actualizado y persistido de forma continua.

A.4.2. Paquete `knowledge_manager`

Implementa los nodos ejecutables encargados de la gestión activa de la información y la resolución de diálogos.

1. **world_state_node**:
   Mantiene en memoria el estado del mundo (robot, ubicaciones, objetos, personas y analítica). Se inicializa a partir de los dominios de `knowledge_bank` y persiste cada cambio en `runtime/world.yaml`. Atiende peticiones en grupos de callbacks independientes protegidos por un *mutex* concurrente de lectura/escritura (permite múltiples lecturas simultáneas sin bloqueo).

- **Servicios ofrecidos**:
  `/knowledge/knowledge_manager/world/get_state` (consulta filtrada por categoría, id y atributo; devuelve un objeto JSON serializado),
  `/knowledge/knowledge_manager/world/update_state` (actualiza atributos de entidades y persiste el estado).

2. **phrase_manager_node**:
   Carga las plantillas de diálogo desde `knowledge_bank/phrases`. Al recibir una solicitud, selecciona una frase aleatoria para la clave especificada y resuelve dinámicamente sus comodines (*placeholders*) con formato `{categoria:atributo:id}` o `categoria_atributo[_id]` consultando al estado del mundo.

   - **Servicios ofrecidos**:
     `/knowledge/knowledge_manager/phrases/get_phrase`.
   - **Servicios utilizados**:
     `/knowledge/knowledge_manager/world/get_state`.

A.4.3. Paquete `knowledge_tools` (Librería cliente)

Librería C++ (clase `Knowledge`) que encapsula la interacción vía clientes de servicio de ROS 2 con `world_state_node` y `phrase_manager_node`. Permite a módulos externos (*Executor*, *Supervisor*, *HRI*) consultar y actualizar el estado del mundo o solicitar frases resueltas sin necesidad de reimplementar la lógica de comunicación RPC ni el parseo de datos.

A.5. HRI

El módulo *HRI* (*Human-Robot Interaction*) gestiona la interacción hablada (síntesis y reconocimiento de voz), la detección y seguimiento de personas, y la interfaz visual de operación del robot. Localiza a la persona objetivo combinando LiDAR (piernas) y visión (cuerpo), mantiene al robot orientado o navegando hacia ella durante el seguimiento, y expone estas capacidades mediante la librería `hri_tools`.

A diferencia del resto del módulo, el paquete `gui` se ejecuta físicamente en la computadora embebida de la base Robotino 4 (repositorio `Robotino`) bajo el *namespace* `/hardware`; se documenta aquí debido a su clasificación funcional en el repositorio y su rol como panel visual que consume tópicos de HRI y Vision.

A.5.1. Paquete `hri_tools` (Librería cliente y launch raíz)

Librería *Facade* (clase `HRI`) que encapsula en llamadas síncronas los servicios asíncronos de Talker, Listener, seguimiento de personas y detección de gestos (utilizada por primitivas como `Say` y `Listen` de `executor_tools`). Aloja el archivo `hri.launch.xml`, el cual lanza bajo el *namespace* `/hri` a Talker, Listener, `human_detector` y `human_follower`.

A.5.2. Paquete `human_detection` (`human_detector`)

Colección de nodos en Python basados en YOLO para detección y seguimiento de personas.

1. **person_tracker**:
   Fusiona las detecciones de YOLO con la nube de puntos 3D de la cámara para estimar la posición tridimensional de la persona objetivo, orientar la cabeza del robot hacia ella y exponer el enfoque de visión como servicio.

   - **Tópicos publicados**:
     `/human_detection/target_point` (`geometry_msgs/PointStamped`),
     `/hri/human_tracking/confidence` (`std_msgs/Float32`),
     `/hardware/head/look_at` (`geometry_msgs/PointStamped`),
     `/human_detection/track_markers` (`visualization_msgs/MarkerArray`),
     `/navigation/camera_obstacles/bypass` (`std_msgs/Bool`).
   - **Tópicos suscritos**:
     `/vision/detections_list`,
     `/hardware/camera/points` (nube de puntos).
   - **Servicios ofrecidos**:
     `/vision/focus`, `/vision/focus/enable`, `/vision/focus/disable`.
   - **Servicios utilizados**:
     `/vision/enable_continuous`.

2. **pose_detector_node** (`pose_detector.py`):
   Nota de despliegue: Corre en el sistema activo consumido por `HRI::FindPose`, pero se lanza desde el módulo *Vision* (`vision.launch.xml`) y no desde `hri.launch.xml`.
   Detecta si una persona está saludando o apuntando con la muñeca.

   - **Tópicos publicados**:
     `/human_detection/pose_image_wrist` (`sensor_msgs/Image`).
   - **Tópicos suscritos**:
     Imagen RGB, nube de puntos y `camera_info` de la cámara.
   - **Servicios ofrecidos**:
     `/human_detection/detect_wrist`.

A.5.3. Paquete `human_follower`

Pipeline de seguimiento de personas que combina la detección de piernas por LiDAR y la posición de cuerpo por visión mediante filtrado estocástico.

1. **leg_finder_node**:
   Procesa el barrido láser de la base para identificar patrones geométricos correspondientes a piernas humanas.

- **Tópicos publicados**:
  `/hri/leg_finder/legs_pose` (`geometry_msgs/PoseArray`),
  `/hri/leg_finder/marker`.
- **Tópicos suscritos**:
  `/hardware/base/scan`,
  `/hri/leg_finder/enable` (`std_msgs/Bool`).

2. **human_follower_nodeV2**:
   Fusiona mediante un filtro de Kalman de 4 estados la posición de las piernas (`leg_finder_node`) y del cuerpo (`person_tracker`), publicando metas de navegación continuas para mantener una distancia fija respecto a la persona.

   - **Tópicos publicados**:
     `/nav_control/goal`,
     `/navigation/stop`,
     `/hri/human_follower/state`,
     `/hri/human_follower/debug_fused_point`.
   - **Tópicos suscritos**:
     `/hri/human_following/enable`,
     `/hri/leg_finder/legs_pose`,
     `/human_detection/target_point`.

3. **human_tracking_node**:
   Coordinador de alto nivel e interruptor principal del seguimiento. Al activarse, habilita simultáneamente la visión continua, la detección de piernas y el seguidor de cuerpo, incorporando un *watchdog* sobre la confianza de la estimación para desactivar el seguimiento si el objetivo se pierde.

   - **Tópicos publicados**:
     `/hri/human_following/enable`,
     `/hri/leg_finder/enable`,
     `/vision/enable_continuous`,
     `/hri/human_tracking/state`.
   - **Tópicos suscritos**:
     `/hri/human_tracking/confidence`.
   - **Servicios ofrecidos**:
     `/hri/human_tracking/enable`.
   - **Servicios utilizados**:
     `/vision/focus/enable`, `/vision/focus/disable`.

A.5.4. Paquete `listener`

Reconocimiento de voz automatizado (ASR).

1. **listener_node**:
   Captura audio local desde el micrófono físico, aplica Detección de Actividad de Voz (VAD) y realiza la transcripción mediante Whisper (restringido opcionalmente por gramáticas JSON según la prueba).

   - **Tópicos publicados**:
     `/hsrb/gui/state`.
   - **Servicios ofrecidos**:
     `/hri/listener/listen`.

A.5.5. Paquete `talker`

Síntesis de voz (TTS).

1. **talker_node**:
   Sintetiza texto a señal de audio utilizando el motor Piper y reproduce la locución a través de los altavoces del robot.

   - **Tópicos publicados**:
     `/hsrb/gui/text`.
   - **Servicios ofrecidos**:
     `/hri/talker/talk`.

A.5.6. Paquete `gui` (Paquete ROS `justina_gui`)

Panel de operación en interfaz gráfica (PyQt) ejecutado en la computadora de la base Robotino 4 (lanzado en `justina_hardware` bajo el *namespace* `/hardware`).

1. **gui_node** (`justina_gui_node`):
   Muestra en pantalla el estado operativo del robot durante pruebas (paro de emergencia, texto actual de síntesis/reconocimiento e imágenes de depuración de visión). Es un nodo únicamente suscriptor/visualizador.

   - **Tópicos suscritos**:
     `/navigation/navigation_ctrl/stop`,
     `/hsrb/gui/state`,
     `/hsrb/gui/text`,
     `/vision/owl/debug_image`,
     `/human_detection/pose_image_wrist`,
     `/face_recog/debug_image`,
     `/vision/detect_seats/debug_image`.

A.6. Vision

El módulo *Vision* gestiona la percepción visual del robot: detección de objetos 2D/3D (YOLO y OWLv2), analítica sobre detecciones (conteo, tamaño, color, comparación), reconocimiento e identificación facial, detección de asientos vacíos, alineación con mesas y localización de espacios de entrega (mesa/repisa/contenedor) para manipulación. Expone estas capacidades mediante la fachada `vision_tools`. Es uno de los módulos con más nodos activos simultáneamente (14, todos bajo el *namespace* `/vision`).

A.6.1. Paquete `vision_tools` (Librería cliente y launch raíz)

Librería *Facade* (clase `Vision`) que encapsula en llamadas síncronas los servicios de detección, analítica y reconocimiento facial. Aloja el archivo `vision.launch.xml` (que configura los parámetros de `vacancy_detector`) y tres nodos propios:

1. **detect_seats_node**:
   Combina detección YOLO y conversión a pose 3D para determinar si hay un asiento libre frente al robot.

   - **Tópicos publicados**:
     `/vision/detect_seats/debug_image` (`sensor_msgs/Image`).
   - **Tópicos suscritos**:
     Imagen de la cámara (parámetro `image_topic`).
   - **Servicios ofrecidos**:
     `/vision/find_seat`.
   - **Servicios utilizados**:
     `/vision/detection`, `/vision/bbox_to_pose`.

2. **Detect_Fusion_Node** (`detect_fusion_node`):
   Selecciona un detector 2D (YOLO u OWLv2), obtiene el *bounding box* de la clase solicitada y lo convierte a pose 3D vía `pc_bbox_pose`.
   Nota de integración: Ofrece el servicio como `/detect_by_class`, mientras que la fachada `Vision::DetectByClass()` invoca `/vision/detect_by_class`; debido a este desfase de prefijo, la llamada síncrona desde la fachada no encuentra servidor en la configuración actual.

   - **Servicios ofrecidos**:
     `/detect_by_class`.
   - **Servicios utilizados**:
     `/vision/detection`, `/vision/owl/owlv2_service`, `/vision/bbox_to_pose`.

3. **image_fixed_node** (`hsr_image_enhancer`):
   Aplica balance de blancos y realce a la imagen de la cámara del HSR/Takeshi. En la plataforma real con base Robotino y cámara `openni2_camera`, este nodo permanece inactivo al no recibir datos en el tópico de origen.

   - **Tópicos publicados**:
     `/hardware/camera/image_raw`.
   - **Tópicos suscritos**:
     `/head_rgbd_sensor/rgb/image_rect_color`.

A.6.2. Paquete `object_recognizer` (Paquetes `yolo_detect`, `owlv2_detect` y `pc_bbox_pose`)

Conjunto de tres paquetes ROS orientados a la detección 2D/3D y analítica visual de objetos.

1. **yolo_detect_node** (paquete `yolo_detect`):
   Ejecuta YOLO11 sobre la imagen de entrada para publicar la lista continua de objetos o responder a peticiones bajo demanda.

   - **Tópicos publicados**:
     `/vision/detections_list` (consumido por `person_tracker` de HRI).
   - **Tópicos suscritos**:
     Imagen de la cámara, `/vision/enable_continuous` (`std_msgs/Bool`).
   - **Servicios ofrecidos**:
     `/vision/detection`, `/vision/enable_continuous`.

2. **owlv2_detect_node** (`owl_server`, paquete `owlv2_detect`):
   Ejecuta el modelo OWLv2 para detección de vocabulario abierto basada en texto (*queries*).

   - **Tópicos publicados**:
     `/vision/detections`, `/vision/owl/debug_image`.
   - **Tópicos suscritos**:
     Imagen de la cámara.
   - **Servicios ofrecidos**:
     `/vision/owl/owlv2_service`.
   - **Servicios utilizados**:
     `/vision/bbox_to_pose`.

3. **Analize_node** (`vision_analytic_service`, paquete `owlv2_detect`):
   Realiza analítica sobre las detecciones de OWLv2 (conteo con seguimiento IoU, filtrado por tamaño, color y comparación).

Nota de despliegue: El paquete instala además `Analize_Human_node` (servidor de `/vision/analyze_person`), pero este no es lanzado en `vision.launch.xml`, dejando sin backend activo a `Vision::AnalyzePerson()`.

- **Tópicos suscritos**:
  `/vision/detections`.
- **Servicios ofrecidos**:
  `/vision/count_objects`, `/vision/get_by_size`, `/vision/get_by_color`, `/vision/compare_count`.
- **Servicios utilizados**:
  `/vision/owl/owlv2_service`.

4. **pc_bbox_pose_node** (paquete `pc_bbox_pose`):
   Procesa un *bounding box* 2D junto con la nube de puntos 3D para estimar la pose tridimensional de un objeto.

   - **Tópicos suscritos**:
     Nube de puntos de la cámara (parámetro `pointcloud_topic`).
   - **Servicios ofrecidos**:
     `/vision/bbox_to_pose`.

A.6.3. Paquete `face_recognizer` (Paquete ROS `face_recog`)

Percepción facial para detección, identificación, descripción y registro de personas.

1. **face_recog_node** (`faceFlorence` / `face_service_node`):
   Implementa el pipeline completo de rostro utilizando el modelo Florence-2. Los ejecutables heredados `face` y `facev2` se conservan en el paquete pero no son lanzados.

   - **Tópicos publicados**:
     `/face_recog/debug_image` (`sensor_msgs/Image`).
   - **Tópicos suscritos**:
     Imagen RGB y `camera_info` de la cámara.
   - **Servicios ofrecidos**:
     `/vision/detect_person`, `/vision/identify_person`, `/vision/describe_person`, `/vision/register_person`.

A.6.4. Paquete `table_alignment`

Estimación de la geometría de superficies planas para aproximación del robot.

1. **table_segmenter_node**:
   Segmenta la nube de puntos para encontrar el plano de una mesa (redonda o rectangular) y calcula el ángulo de alineación con su borde.

   - **Tópicos publicados**:
     Nube de puntos segmentada, ángulo de alineación (`std_msgs/Float64`), marcadores de visualización para RViz.
   - **Tópicos suscritos**:
     Nube de puntos de la cámara.
   - **Servicios ofrecidos**:
     `/vision/get_table_alignment`.

A.6.5. Paquete `vacancy_detector`

Localización de zonas despejadas para tareas de manipulación y depósito sobre mesas, repisas o contenedores.

1. **vacancy_cloud_filter** (`vacancy_cloud_filter_node`):
   Filtro ROI compartido que recorta la nube de puntos y suspende temporalmente los filtros de navegación mientras la detección de espacio está activa.

   - **Tópicos publicados**:
     `/vision/vacancy/points_filtered`, habilitación de campos potenciales en navegación.
   - **Tópicos suscritos**:
     Nube de puntos de la cámara.
   - **Servicios ofrecidos**:
     `/vision/vacancy_cloud_filter/enable`.
   - **Servicios utilizados**:
     Servicio de habilitación del filtro de obstáculos de *Navigation*.

2. **table_drop_tf_node** / **shelf_drop_tf_node** / **container_drop_tf_node**:
   Calculan puntos libres para la colocación de objetos en mesas, estantes (detectando planos horizontales) y contenedores (acotando la región vía OWLv2). Publican los marcos de referencia resultantes como transformaciones TF (`table_frame`, `shelf_frame`, `container_frame`) y sus aproximaciones (`*_drop_start`).

   - **Tópicos publicados**:
     Marcadores de depuración (RViz); nube recortada de depuración en `container_drop_tf_node`
   - **Tópicos suscritos**:
     `/vision/vacancy/points_filtered`.
   - **Servicios ofrecidos**:
     `enable` (`std_srvs/SetBool`, individual por nodo).

- **Servicios utilizados**:
  `/vision/vacancy_cloud_filter/enable`; `/vision/owl/owlv2_service` (únicamente `container_drop_tf_node`).

A.7. Navigation

El módulo *Navigation* se encarga de localizar al robot, mantener un mapa de obstáculos dinámicos superpuesto al mapa estático, planear rutas sobre ese mapa y ejecutarlas con evitación reactiva de obstáculos. Es el módulo con más paquetes del repositorio (14), aunque no todos forman parte del sistema activo de Justina: varios existen específicamente para el backend alterno del robot Takeshi (HSR) o para la simulación en Webots, y se documentan como tales.

Arquitectura en capas (sistema activo): `nav_controller_node` y `locations_manager_node` constituyen la interfaz pública (usada por `navigation_tools` y por el resto del sistema) y corren en el *namespace* raíz, no bajo `/navigation`. Por debajo, `mvn_pln` (`motion_planner_node`) orquesta una maniobra completa hacia una meta, apoyándose en `path_planner` (ruta A* sobre el mapa aumentado), `simple_move` (seguimiento de trayectoria/control de velocidad) y `potential_fields` (evitación reactiva superpuesta al `cmd_vel`). El mapa aumentado lo mantienen `map_augmenter` y `obstacle_detector` (obstáculos dinámicos por LiDAR/nube) a partir del mapa estático servido por `nav2_map_server` y localizado por `nav2_amcl` (modo AMCL) o `slam_toolbox` (modo SLAM/`mapless`, compartiendo el resto de la pila).

A.7.1. Paquete `navigation_tools` (Librería cliente y launch)

Fachada síncrona (clase `Navigation`) sobre las tareas de navegación a ubicaciones conocidas y el registro de la pose actual como una ubicación nueva. Aloja también un *launch* propio, aunque el archivo principal del robot (`justina.launch.xml`) incluye directamente a `navigation_start`.

A.7.2. Paquetes de la interfaz de control (`navigation_controller`)

Proporcionan la interfaz pública del módulo. Ambos nodos operan en el *namespace* raíz.

1. **locations_manager_node** (paquete `locations_manager`):
   Guarda y consulta ubicaciones nombradas (`config/known_locations.yaml`) y difunde su transformación TF periódicamente para visualización.

   - **Tópicos publicados**:
     Transformaciones TF de cada ubicación conocida (frecuencia de 0.5s, para visualización en RViz).

- **Servicios ofrecidos**:
  `/locations_manager/add`, `/locations_manager/get`, `/locations_manager/save`.

2. **nav_controller_node** (paquete `nav_controller`):
   Traduce las instrucciones de ïr a ubicación conocida.[en] una meta de pose concreta, calcula un *timeout* proporcional a la distancia a recorrer y difunde la señal de paro de navegación consumida por los *drivers* de hardware.

   - **Tópicos publicados**:
     `/nav_control/goal`, `/navigation/navigation_ctrl/stop`.
   - **Tópicos suscritos**:
     `/navigation/status`.
   - **Servicios ofrecidos**:
     `/nav_controller/go_to_location`.
   - **Servicios utilizados**:
     `/locations_manager/get`.

A.7.3. Paquete de agrupación `navigation_planner`

Conjunto de nueve paquetes ROS (siete propios y dos exclusivos del backend Takeshi) dedicados a la planeación, control reactivo y procesamiento de mapas.

1. **navigation_start**:
   Paquete de configuración, mapas y *launch* (sin nodos propios). Aloja `navigation_justina_amcl.l`
   (modo AMCL por defecto), `navigation_justina_slam.launch.xml` (modo SLAM/`mapless`
   vía `slam_toolbox`), mapas estáticos y de prohibición por escenario (incluye RoboCup 2026 y TMR 2026), y las variantes para el backend Takeshi/HSR no utilizadas por Justina.

2. **motion_planner_node** (`mvn_pln`):
   Orquesta la maniobra completa de navegación hacia una meta: solicita la ruta a `path_planner`, la despacha a `simple_move`, coordina `potential_fields` y reacciona a paros y eventos de riesgo de colisión.

   - **Tópicos publicados**:
     `/navigation/status`, `/navigation/potential_fields/enable`, `/navigation/potential_`
     `/simple_move/goal_path`, `/simple_move/goal_dist_angle`, `/simple_move/stop`.
   - **Tópicos suscritos**:
     `/nav_control/goal`, `/stop`, `/navigation/stop`, `/simple_move/goal_reached`,
     `/navigation/set_patience`, `/navigation/potential_fields/collision_risk`.
   - **Servicios utilizados**:
     `path_planner/plan_path_with_static`, `path_planner/plan_path_with_augmented`,
     y servicios de consulta e información de `map_augmenter`.

3. **path_planner_node** (paquete `path_planner`):
   Búsqueda de rutas punto a punto utilizando el algoritmo A* (con opción de movimientos diagonales) sobre el mapa estático o el mapa aumentado.

   - **Servicios ofrecidos**:
     `path_planner/plan_path_with_static`, `path_planner/plan_path_with_augmented`.
   - **Servicios utilizados**:
     Servicios `GetMap` expuestos por `map_augmenter`.

4. **simple_move_node** (paquete `simple_move`):
   Convierte rutas u objetivos en comandos de velocidad mediante un control proporcional lineal/angular, con capacidad opcional de orientar la cabeza en la dirección de avance.

   - **Tópicos publicados**:
     `/hardware/base/cmd_vel` (vía *remap* de `/cmd_vel`), `/simple_move/goal_reached`, comandos de orientación de cabeza.
   - **Tópicos suscritos**:
     `/simple_move/goal_path`, `/simple_move/goal_dist_angle`, `/simple_move/stop`, `/stop`, señal de paro de navegación, y vector de repulsión de `potential_fields`.

5. **potential_fields_node** (paquete `potential_fields`):
   Calcula un campo de repulsión reactivo a partir de datos de LiDAR y nube de puntos, superponiéndolo al `cmd_vel` de `simple_move` para evitar colisiones dinámicas no mapeadas. Incluye el script auxiliar `publish_enable.py`.

   - **Tópicos publicados**:
     `/hardware/base/cmd_vel` (modificación de velocidad comandada), `/navigation/potential`
   - **Tópicos suscritos**:
     `/navigation/potential_fields/enable`, `/navigation/potential_fields/enable_cloud`
     `/simple_move/goal_path`, LiDAR y nube de puntos.

6. **map_augmenter_node** (paquete `map_augmenter`):
   Combina el mapa estático, el mapa de prohibición y los obstáculos dinámicos en un "mapa aumentado"(y su representación de costos) para la planeación de rutas.
   Nota de integración: El paquete compila además `cloud_filter_node`, pero este permanece comentado en el *launch* activo.

   - **Tópicos publicados**:
     Mapa aumentado (`nav_msgs/OccupancyGrid`) y mapa de planeación.
   - **Tópicos suscritos**:
     `clicked_point` (vía `map_enhancer`), mapa de obstáculos dinámicos.

- **Servicios ofrecidos**:
  Cuatro variantes del servicio `GetMap` (estático, costo estático, aumentado, costo aumentado), `are_there_obstacles`, `is_inside_obstacles`.
- **Servicios utilizados**:
  Servicios `GetMap` del servidor de mapa estático y del mapa de prohibición.

7. **obstacle_detector_node** (paquete `map_augmenter`):
   Detecta obstáculos dinámicos a partir del LiDAR y la nube de puntos filtrando por una ventana espacial configurable, publicando un `OccupancyGrid` con decaimiento temporal.

   - **Tópicos publicados**:
     Mapa de obstáculos dinámicos (`nav_msgs/OccupancyGrid`).
   - **Tópicos suscritos**:
     LiDAR y nube de puntos de la cámara.
   - **Servicios utilizados**:
     `GetMap` del servidor de mapa estático.

8. **augment_gridmap_online_node** (`map_enhancer`, paquete `augment_gridmap_online`):
   Herramienta interactiva para añadir obstáculos permanentes al mapa de prohibición mediante clics de selección en RViz.

   - **Tópicos publicados**:
     Mapa de prohibición aumentado, metadatos y marcador de depuración.
   - **Tópicos suscritos**:
     `clicked_point`, mapa de prohibición base.
   - **Servicios ofrecidos**:
     `Empty` (limpiar mapa) y `GetMap` (remapeado a `prohibition_map`).

9. **hardware_controller** (**no usado por Justina**):
   Contiene `head_controller` y `arm_controller`. Traduce entre la convención de Justina y la interfaz de ROS 2 `JointTrajectoryController`. Solo se referencia desde los archivos de lanzamiento orientados al backend Takeshi/HSR.

10. **map_server_bypass** (**no usado por Justina**):
    Puentea el mapa de Google Cartographer hacia la interfaz `GetMap` estándar. Solo se utiliza en la pila de navegación del robot Takeshi.

A.7.4. Paquetes de navegación sin mapa (`mapless_navigation`)

Módulo compuesto por dos paquetes (`mapless_nav` y `debugger`) que **no se ejecutan en el robot real**.

1. **mapless_nav**:
   Implementa navegación reactiva pura sin mapa global mediante campos potenciales, un árbol de comportamiento (`bt_navigation.py`) y un modelo recurrente entrenado (`RNN_mapless.py` + `TorchModels/RNN.pth`). Únicamente se lanza en el entorno de simulación Webots (`justina_mapless_wbt.launch.xml`).

2. **debugger**:
   Librería cliente (`DebugClient`) para el envío de mensajes de estado desde cualquier nodo hacia una consola unificada en terminal (`debug_manager.py`). Se ejecuta manualmente para tareas de depuración y no es invocado por ningún archivo *launch*.

A.8. Manipulation

El módulo *Manipulation* se encarga de planear y ejecutar las trayectorias de brazos y torso necesarias para tomar, colocar y manipular objetos. Actúa como puente entre los consumidores de manipulación del robot (por ejemplo, la FSM `Take` del Executor) y MoveIt2, que se usa únicamente como motor de planeación cinemática y de colisiones (IK vía TRAC-IK, planeación con OMPL/RRTConnect), sin controlar el hardware directamente. La ejecución real sobre los servomotores se delega a los nodos de `Hardware` (`arms_node`, controlador de torso).

A.8.1. Paquete `manipulation_tools` (Librería cliente y launch)

Proporciona las utilidades comunes de la pila de manipulación. Incluye la librería cliente `Manipulation` (patrón *Facade*), que expone las funciones `PlanTrajectory` y `FollowTrajectory` como llamadas síncronas para simplificar la integración con otros módulos (como `executor_tools`). Aloja además el archivo de lanzamiento principal (`manipulation.launch.xml`), que levanta bajo el *namespace* `/manipulation` los nodos `move_group` (MoveIt2), `manipulation_planner_node`, `manipulation_controller_node` y `manipulation_cloud_filter_node`, remapeando la lectura del estado articular a la fuente canónica `/hardware/joint_states`.

A.8.2. Paquetes de planeación y control

Contienen la lógica principal de interconexión con MoveIt2 y la ejecución de trayectorias en hardware.

1. **manipulation_planner_node** (paquete `manipulation_planner`):
   Nodo puente entre los consumidores de manipulación y MoveIt2. Recibe una pose objetivo del efector final, valida el grupo de planeación y el *tip link*, lee el estado articular real desde `/hardware/joint_states`, y solicita la planeación a MoveIt2

forzando el pipeline OMPL con el algoritmo RRTConnect. Opcionalmente coordina el filtro de nube de manipulación, limpia el *octomap* y excluye el objeto objetivo para evitar que sea tratado como obstáculo durante la fase de aproximación.

- **Tópicos suscritos**:
  `/hardware/joint_states` (estado articular real del robot, fuente canónica para el punto de partida).
- **Servicios ofrecidos**:
  `/manipulation/manipulation_planner/plan_trajectory`.
- **Servicios utilizados**:
  `/manipulation/plan_kinematic_path` (solicitud `GetMotionPlan` a `move_group`), `/manipulation/cloud_filter/enable`, `/manipulation/clear_octomap`.

2. **manipulation_controller_node** (paquete `manipulation_controller`):
   Ejecuta una trayectoria articular previamente planeada. Recibe la lista de *waypoints* (bloques compuestos por posiciones de torso y 6 articulaciones del brazo correspondiente) y los envía en paralelo a los nodos de hardware, esperando confirmación bloque por bloque.

   - **Servicios ofrecidos**:
     `/manipulation/manipulation_controller/follow_trajectory`.
   - **Servicios utilizados**:
     `/hardware/arms/move_right_arm`, `/hardware/arms/move_left_arm`, `/hardware/torso/mo`

3. **manipulation_cloud_filter_node** (paquete `manipulation_tools`):
   Recorta la nube de puntos cruda de la cámara a una región de interés (ROI) acotada al volumen útil de los brazos y aplica una reducción por *voxelización*. Publica la nube procesada para alimentar el *octomap* de MoveIt2, evitando interferencias con el suelo o zonas fuera del alcance. Permanece desactivado por defecto y se habilita bajo demanda para optimizar el cómputo.

   - **Tópicos publicados**:
     `/manipulation/camera/points_filtered`, `/manipulation/cloud_filter/debug_markers` (marcadores opcionales de ROI/voxelización).
   - **Tópicos suscritos**:
     `/hardware/camera/points` (nube cruda de la cámara).
   - **Servicios ofrecidos**:
     `/manipulation/cloud_filter/enable`.

A.8.3. Paquetes de soporte y dependencias (`move_it` y `trac_ik`)

Paquetes complementarios para la configuración cinemática y solvers de cinemática inversa.

1. **move_it**:
   Paquete de configuración generado mediante *MoveIt Setup Assistant* adaptado a la cinemática de Justina. Contiene las definiciones URDF/SRDF, los grupos de planeación (`left_arm_group`, `right_arm_group`), la configuración de límites articulares y el archivo `kinematics.yaml` asociado al solver TRAC-IK. Se encarga de levantar el nodo `move_group` de MoveIt2.

2. **trac_ik**:
   Librería vendorizada de terceros (TRACLabs Robotics) utilizada como *plugin* de cinemática inversa (IK) dentro de MoveIt2. No genera nodos propios del proyecto.

A.9. Hardware

El módulo *Hardware* se encarga de exponer el hardware físico del robot (cabeza, torso, brazos, pinzas, base omnidireccional, cámara RGB-D y LiDAR) como una fachada de ROS 2 síncrona para el resto del sistema, y de controlar cada uno de esos actuadores/sensores a bajo nivel.

Arquitectura distribuida: el sistema de Justina corre en dos computadoras que forman un mismo grafo de ROS 2 (descubrimiento DDS/CycloneDDS sobre una red Ethernet dedicada), ambas bajo el *namespace* común `/hardware`:

- La **computadora principal** ejecuta `hardware_tools` (la fachada `Hardware`), `door_detector`, `scanner` (puente de orientación de cabeza), `kinect_ros2` y el *driver* de cámara RGB-D (`openni2_camera`), además de todos los módulos de software de más alto nivel (Executor, HRI, Knowledge, Manipulation, Navigation, Planning, Supervisor, Vision).

- La **computadora embebida en la base Robotino 4 (FESTO)** (repositorio `Robotino`) ejecuta los *drivers* de bajo nivel de la base, cabeza, torso, brazos y pinzas (`justina_hardware`, `base`, `head`, `torso`, `arms`, `grippers`, `joint_states_parser`) y el panel de operación (`gui`, documentado en la sección de *HRI* por la categorización del propio repositorio).

El paquete `justina_description` está duplicado byte a byte en ambos repositorios (cada computadora necesita el URDF para su propio `robot_state_publisher` / TF local), por lo que se documenta una sola vez. Los dos repositorios son consistentes entre sí: los nombres de tópicos y servicios que `hardware_tools/Hardware.hpp` espera del lado de la base Robotino (p. ej. `/hardware/arms/move_right_arm`, `/hardware/torso/current_pose`) coinciden exactamente con los que los nodos de `Robotino` realmente exponen.

A.9.1. Paquete `hardware_tools` (Librería cliente y launch raíz)

Librería *Facade* (clase `Hardware`) que encapsula en llamadas síncronas los servicios de torso, cabeza, brazos, pinzas, base y detección de puerta, para que otros módulos no

manejen clientes asíncronos de ROS 2 directamente. Aloja el archivo `hardware.launch.xml` (que levanta `openni2_camera`, `depth_image_proc`, `door_detector` y `scanner`, así como condicionalmente el *bringup* de Takeshi/HSR) y administra la comunicación síncrona del módulo.

A.9.2. Paquetes de percepción de la computadora principal (`door_detector`, `scanner`, `kinect_ros2` y `justina_description`)

Conjunto de paquetes alojados en la computadora principal dedicados al procesamiento del LiDAR para puertas, la orientación de la cabeza hacia puntos 3D y el modelado/simulación del robot.

1. **door_detector_node** (paquete `door_detector`): Determina si una puerta está abierta o cerrada comparando la distancia promedio del sector central del barrido láser de la base contra un umbral (0.75,m).
   - **Tópicos suscritos**:
     `/hardware/base/scan` (`sensor_msgs/LaserScan`).
   - **Servicios ofrecidos**:
     `/hardware/door_detector/check`.
2. **head_control_node** (paquete `scanner`):
   Puente entre un punto 3D de interés (publicado por `Hardware::LookAt`) y el objetivo de cabeza: calcula pan/tilt vía TF entre el marco de la cámara/cuello y el objetivo, respetando límites articulares y una zona muerta para evitar micromovimientos. Reacciona además al estado de navegación para evitar mover la cabeza en fases críticas.
   - **Tópicos publicados**:
     `/hardware/head/goal_pose` (`std_msgs/Float32MultiArray`).
   - **Tópicos suscritos**:
     `/hardware/head/look_at` (`geometry_msgs/PointStamped`), `/hardware/head/current_pos` (`std_msgs/Float32MultiArray`), `/navigation/status` (`action_msgs/GoalStatusArray`).
3. **kinect_ros2**:
   Controlador de terceros (basado en `libfreenect`) para la cámara Kinect v1, vendorizado en el árbol de código. No forma parte del *pipeline* activo de cámara (que usa `openni2_camera`); sólo se referencia desde *launches* de depuración.
4. **simple_justina_sim** (nodo del paquete `justina_description`):
   Simulador ligero en software utilizado únicamente en configuración de pruebas sin robot físico (`justina_sim.launch.xml`). Sustituye a `hardware_tools` y a los *drivers* reales: integra la velocidad comandada, publica odometría, TF `odom a base_link`, articulaciones falsas y un láser simulado, e implementa directamente los servicios de movimiento de cabeza, torso y brazos.

- **Tópicos publicados**:
  /hardware/base/odom (nav_msgs/Odometry), /hardware/joint_states (sensor_msgs/Joi
  /hardware/base/scan (sensor_msgs/LaserScan), /hardware/head/current_pose
  (std_msgs/Float32MultiArray), /hardware/torso/current_pose (std_msgs/Float32Mul
  /hardware/arms/left_arm/current_pose (std_msgs/Float32MultiArray),
  /hardware/arms/right_arm/current_pose (std_msgs/Float32MultiArray),
  /navigation/initialpose (geometry_msgs/PoseWithCovarianceStamped).
- **Tópicos suscritos**:
  /hardware/base/cmd_vel (geometry_msgs/Twist), /hardware/head/goal_pose
  (std_msgs/Float32MultiArray), /hardware/torso/goal_pose (std_msgs/Float32MultiA
  /hardware/arms/left_arm/goal_pose (std_msgs/Float32MultiArray), /hardware/arms/
  (std_msgs/Float32MultiArray), /initialpose (geometry_msgs/PoseWithCovarianceSta
  /navigation/initialpose (geometry_msgs/PoseWithCovarianceStamped).
- **Servicios ofrecidos**:
  /hardware/head/move_head, /hardware/torso/move_torso, /hardware/arms/move_left_
  /hardware/arms/move_right_arm.

5. **kinect_tilt_motor_node** (nodo del paquete justina_description):
   Refleja el *tilt* de cabeza comandado hacia el motor físico de inclinación del Kinect v1 vía USB; sólo se lanza de forma opcional cuando la variable use_kinect está activa.

   - **Tópicos suscritos**:
     /hardware/head/current_pose (std_msgs/Float32MultiArray).

6. **depth_to_pointcloud** (virtual_pointcloud.cpp, paquete justina_description):
   Utilidad de depuración que convierte una imagen de profundidad y su camera_info en una nube de puntos (sensor_msgs/PointCloud2). No forma parte del *pipeline* principal de percepción.

A.9.3. Paquetes de bajo nivel en la computadora Robotino (Robotino)

Conjunto de paquetes que ejecutan directamente en la computadora de la base Robotino 4 para interactuar con los servomotores, controladores seriales, microcontroladores por red Ethernet y la API del fabricante.

1. **justina_hardware**:
   Bringup raíz del lado Robotino. Bajo el *namespace* /hardware, remapea los tópicos joint_states relativos de cada driver al tópico canónico /hardware/joint_states, incluye los *launches* de los subcomponentes, ejecuta la GUI del panel de operación y levanta un robot_state_publisher local con entrada remapeada a /manipulation/joint_states.

2. **base_node** (nodo `RTONode`, paquete `base`):
   Driver principal de la base Robotino 4. Conecta por TCP al servidor *RobotinoAPI2* del fabricante (clase `ComROS`) para exponer tracción omnidireccional, parachoques (*bumper*), lecturas de motores, gestión de energía, IMU, entradas/salidas digitales y encoders.

   - **Tópicos publicados**:
     `/hardware/base/bumper` (`std_msgs/Bool`), `/hardware/base/motor_readings` (`std_msgs/Float32MultiArray`), `/hardware/base/motor_error_readings` (`std_msgs/Float32MultiArray`), `/hardware/base/joint_states` (`sensor_msgs/JointSta`
     `/hardware/base/battery_status` (`sensor_msgs/BatteryState`), `/hardware/base/imu` (`sensor_msgs/Imu`), `/hardware/base/digital_readings` (`std_msgs/BoolMultiArray`), `/hardware/base/encoder_readings` (`std_msgs/Int32MultiArray`).
   - **Tópicos suscritos**:
     `/hardware/base/cmd_vel` (`geometry_msgs/Twist`), `/hardware/base/set_digital_values` (`std_msgs/BoolMultiArray`).
   - **Servicios ofrecidos**:
     `/hardware/base/cmd_vel_enable`, `/hardware/base/set_velocity_limits`, `/hardware/base/set_encoder_position`.

3. **robotino_odometry_node** (paquete `base`):
   Calcula y publica la odometría de la base a partir de las lecturas del driver, operando como un ejecutable independiente del nodo principal.

   - **Tópicos publicados**:
     `/hardware/base/odom` (`nav_msgs/Odometry`); difunde también la TF `odom` a `base_link` si `launch_odom_tf` está activo.
   - **Servicios ofrecidos**:
     `/hardware/base/reset_odometry`.

4. **digital_stop_node** (ejecutable `Stop_node`, paquete `base`):
   Traduce el estado de la entrada digital física (índice 6 del arreglo, conectada a un botón/interruptor de paro) a la señal de alto que consumen navegación, brazos, cabeza y pinzas.

   - **Tópicos publicados**:
     `/navigation/navigation_ctrl/stop` (`std_msgs/Bool`).
   - **Tópicos suscritos**:
     `/hardware/base/digital_readings` (`std_msgs/BoolMultiArray`).

5. **head_node** (paquete `head`):
   Controla los 2 servomotores Dynamixel de pan/tilt de la cabeza vía RS-485 mediante el SDK de Dynamixel.

- **Tópicos publicados**:
  /hardware/head/joint_states (sensor_msgs/JointState), /hardware/head/current_po
  (std_msgs/Float32MultiArray), /hardware/head/goal_reached (std_msgs/Bool).
- **Tópicos suscritos**:
  /hardware/head/goal_pose (std_msgs/Float32MultiArray), /navigation/navigation_c
  (std_msgs/Bool).
- **Servicios ofrecidos**:
  /hardware/head/move_head.

6. **torso_node** (paquete torso):
   Controla el eje lineal del torso mediante un controlador Jrk (Pololu) sobre puerto serial (/dev/torso), usando retroalimentación analógica de posición con tolerancia y verificación de lecturas estables.

   - **Tópicos publicados**:
     /hardware/torso/joint_states (sensor_msgs/JointState), /hardware/torso/current_
     (std_msgs/Float32MultiArray), /hardware/torso/goal_reached (std_msgs/Bool).
   - **Tópicos suscritos**:
     /hardware/torso/goal_pose (std_msgs/Float32MultiArray).
   - **Servicios ofrecidos**:
     /hardware/torso/move_torso.

7. **arms_node** (left_arm_node / right_arm_node, paquete arms):
   Controla los 6 servomotores Dynamixel de cada brazo sobre bus RS-485. El mismo ejecutable se instancía dos veces (parametrizado por left_arm.yaml y right_arm.yaml) e incluye un esquema de compensación de gravedad mediante *offsets* estáticos.

   - **Tópicos publicados**:
     /hardware/arms/left_arm/joint_states y /hardware/arms/right_arm/joint_states
     (sensor_msgs/JointState), /hardware/arms/left_arm/current_pose y
     /hardware/arms/right_arm/current_pose (std_msgs/Float32MultiArray),
     /hardware/arms/left_arm/goal_reached y /hardware/arms/right_arm/goal_reached
     (std_msgs/Bool).
   - **Tópicos suscritos**:
     /hardware/arms/left_arm/goal_pose y /hardware/arms/right_arm/goal_pose
     (std_msgs/Float32MultiArray), /navigation/navigation_ctrl/stop (std_msgs/Bool).
   - **Servicios ofrecidos**:
     /hardware/arms/move_left_arm, /hardware/arms/move_right_arm.

8. **grippers_node** (left_gripper_node / right_gripper_node, paquete grippers):
   Controla las pinzas mediante comunicación TCP/IP hacia un microcontrolador

RP2350 (con interfaz Ethernet W5500) ejecutando un protocolo binario propio de 8 bytes. Se instancía dos veces (una por lado).

- **Tópicos publicados**:
  `/hardware/grippers/left_gripper/joint_states` y `/hardware/grippers/right_grippe`
  (`sensor_msgs/JointState`).
- **Tópicos suscritos**:
  `/navigation/navigation_ctrl/stop` (`std_msgs/Bool`).
- **Servicios ofrecidos**:
  `/hardware/grippers/move_left_gripper`, `/hardware/grippers/move_right_gripper`.

9. **joint_states_parser_node** (paquete `joint_states_parser`):
   Fusiona las lecturas parciales de articulaciones provenientes de los distintos drivers (base, torso, brazos, pinzas, cabeza) publicadas en `/hardware/joint_states` y genera un mensaje completo en orden canónico para MoveIt2.

   - **Tópicos publicados**:
     `/manipulation/joint_states` (`sensor_msgs/JointState`).
   - **Tópicos suscritos**:
     `/hardware/joint_states` (`sensor_msgs/JointState`).

10. **urg_node**:
    Paquete de terceros (*driver* Hokuyo) que lee el LiDAR 2D de la base por serial (`dev/lidar`) acotado a 180 grados y publica la lectura procesada.

    - **Tópicos publicados**:
      `/hardware/base/scan` (`sensor_msgs/LaserScan`, remapeado desde su salida por defecto).

Apéndice B

Protocolo de investigación

Entrevista a usuario final

1. ¿En qué tareas del hogar le gustaría que un robot lo asista?
2. ¿Hay actividades que preferiría hacer usted mismo en lugar de delegarlas a un robot? ¿Por qué?
3. ¿Qué tanto espera que el robot trabaje de forma autónoma en lugar de recibir órdenes explícitas?
4. ¿Cuánto tiempo cree que debería tomarle a un robot realizar sus tareas respecto al tiempo que le tomaría a una persona?
5. Si usted saliera de viaje y el robot se quedara solo en su casa, ¿qué le pediría que hiciera en su ausencia?
6. ¿De qué manera le gustaría poder dar órdenes al robot (escritas, habladas, desde un control, una computadora)?
7. ¿De qué forma se imagina que sería más fácil confirmar que el robot realmente entendió la órden que está por ejecutar?
8. ¿Le gustaría que el robot le reportara el estatus de las actividades encargadas? ¿En qué forma le gustaría que lo hiciera?
9. ¿Qué tan importante le parece que el robot parezca una persona?
10. ¿Cree que el robot debería cambiar su comportamiento dependiendo de con quién interactúa? En caso afirmativo, ¿de qué forma?
11. ¿Qué artículo del hogar le daría más miedo permitir que un robot manipule?
12. ¿Cuáles son los objetos más pesados o frágiles sería útil para usted que el robot pudiera manipular?

13. Además de objetos y bolsas, ¿qué otro tipo de cosas crees que sería útil que el robot pudiera manipular? Ej. Puertas, llaves, cepillos

14. ¿Le gustaría que el robot incluyera un manual para que usted mismo le pudiera dar mantenimiento y hacer reparaciones básicas o preferiría llevarlo con el proveedor directamente ante cualquier falla?

15. ¿Utilizaría el robot únicamente en su hogar o consideraría llevarlo consigo para que lo asista en viajes?

16. ¿La apariencia de qué robot le inspira mayor confianza? ¿Por qué?



(a) Opción 1



(b) Opción 2



(c) Opción 3

17. ¿Cuáles son las principales preocupaciones que le generaría tener un robot de servicio en casa?

18. ¿Qué elementos podría incluir el robot para hacerlo sentir más seguro?

Entrevista al desarrollador de software (General)

1. Cuéntanos acerca de tu experiencia en la RoboCup, enfocada principalmente a la categoría @Home.

2. ¿Cuáles han sido los problemas más importantes que has enfrentado o te has enterado que han enfrentado otros equipos durante una competencia?

3. ¿Cuál consideras que es el hardware mínimo que se requiere para participar en esta categoría? ¿Cuál de esos elementos consideras que tiene un mayor impacto en el desempeño del robot?

4. ¿Consideras que la apariencia del robot afecta la manera en la que el usuario interactúa con él? ¿Crees que este efecto tiene un impacto real en su desempeño?

5. Además de las instrucciones que se le dan a un usuario no experto antes de una prueba, ¿qué otros elementos integrados en el robot has notado que hacen más intuitiva la interacción?

6. ¿Qué otras formas de interacción se han usado cuando el usuario no domina el idioma inglés?

7. ¿Has identificado alguna característica del diseño de los manipuladores, que nosotros u otros equipos hayan implementado, que pudiera incomodar/lastimar al usuario?

8. ¿Qué tareas de manipulación consideras más críticas en la competencia?

9. ¿Cuál consideras una mejor característica al tomar objetos: la velocidad o la precisión? ¿Por qué?

10. ¿Cuál has identificado como el principal motivo por el que a un robot se le caen objetos que ya había tomado?

11. ¿Cuál es el objeto más grande o pesado que se debe tomar en las pruebas?

12. ¿Cuál es el objeto más pequeño y delicado que se ha tenido que manipular en las pruebas?

13. ¿Hay alguna otra característica, además del peso y tamaño, que haga complicado tomar un objeto? (Ej. Rugosidad de la superficie)

14. ¿Qué condiciones de luz has identificado que afectan más a los sistemas de percepción del robot?

15. ¿Has identificado problemas con puntos ciegos del robot en la competencia?

16. ¿Consideras que la medición de par o fuerza en las articulaciones del brazo y gripper sería útil?

17. ¿Qué medidas de redundancia crees que sean más importantes para la seguridad y el rendimiento del robot?

18. ¿Qué elementos de seguridad, además del botón de paro, consideras que es importante incluir en el diseño del robot?

Entrevista al desarrollador de software (Plataforma experimental)

1. ¿Cuáles consideras que son las características clave del manipulador para tomar objetos y cuáles pueden ser compensadas con la posición de la base?

2. ¿Cuál recuerdas como el efector final más efectivo en la competencia?

3. ¿Crees que la integración de una cámara en el gripper mejoraría su habilidad de tomar objetos?

4. ¿Hay alguna característica del robot que consideres importante como el peso o el tamaño? Más allá de las limitaciones que impone el reglamento.

5. En caso de contar con un robot de dos brazos, ¿consideras que deberían ser brazos idénticos o cada brazo debería tener una habilidad diferente? ¿El efector final debería ser intercambiable?

6. En los diseños previos, ¿qué partes del hardware eran más inaccesibles para su mantenimiento?

Entrevista al desarrollador de software (Plataforma comercial)

1. ¿Cuáles consideras que son las mayores virtudes del hardware de Takeshi?

2. Si pudieras modificar algo, ¿lo harías? ¿qué sería?

3. ¿Crees que la integración de la cámara en el gripper mejora su habilidad de tomar objetos o sólo incrementa el tiempo de procesamiento? ¿Sí la utilizan?

4. ¿Considerarías útil que el efector final fuera intercambiable?

5. ¿Qué tareas de manipulación consideras que son las más desafiantes considerando que Takeshi solo cuenta con un brazo?

6. ¿Han tenido que compensar las limitaciones de los grados de libertad que proporciona Takeshi? ¿Cómo lo han hecho?

7. ¿Han tenido que compensar las limitaciones de los grados de libertad que proporciona Takeshi? ¿Cómo lo han hecho?

8. ¿Existe alguna limitación en el mantenimiento del hardware considerando que se trata de un robot comercial?

9. Dado que todos tienen el mismo hardware, ¿qué utilizas como diferenciador?

Riesgos y beneficios anticipados

Riesgos

Esta investigación presenta riesgos mínimos para los participantes. No se utiliza instrumento alguno o contacto con el robot de ningún tipo y únicamente involucra preguntas sobre experiencias técnicas y necesidades cotidianas.

Los riesgos identificados incluyen:

- Posible incomodidad al compartir información sobre necesidades personales de asistencia doméstica.

- Fatiga durante la sesión de entrevista (mitigado mediante sesiones de duración razonable con pausas si es necesario).
- Riesgo mínimo de identificación indirecta a pesar de la codificación de datos.

Beneficios

Los participantes podrán obtener los siguientes beneficios:

- Contribución directa al desarrollo de tecnología de asistencia robótica que puede mejorar la calidad de vida de personas con necesidades similares.
- Oportunidad de reflexionar sobre sus experiencias y necesidades, lo cual puede generar aprendizajes valiosos para su propio contexto.
- Retroalimentación que puede enriquecer su perspectiva sobre el diseño centrado en el usuario.
- Participación en investigación que busca cerrar la brecha entre el desarrollo tecnológico y las necesidades reales de los usuarios finales.

Criterios de selección

Los participantes serán seleccionados mediante muestreo intencional conforme a dos perfiles específicos:

Perfil 1: Equipo de desarrollo

- Ser miembro activo o previo del equipo de desarrollo del robot de servicio doméstico Justina.
- Contar con experiencia de participación en la competencia RoboCup@HOME.
- Tener conocimiento técnico sobre el diseño, programación o implementación de funcionalidades del robot.

Perfil 2: Usuarios potenciales

- Ser personas que requieren asistencia doméstica constante debido a edad avanzada (65 años o más) o alguna condición de discapacidad que limite su autonomía en actividades cotidianas del hogar.
- Residir de forma independiente o semi-independiente en un entorno doméstico.
- Tener capacidad cognitiva para participar en una entrevista y proporcionar consentimiento informado (o contar con representante legal en caso necesario).

Privacidad y confidencialidad

Los datos personales recolectados durante la investigación serán utilizados exclusivamente con fines académicos. Se garantizará la privacidad de los participantes mediante consentimiento informado, y la confidencialidad de la información será protegida a través de la codificación de datos, almacenamiento seguro y acceso restringido únicamente al equipo investigador.

Apéndice C

Carta de consentimiento informado

Por medio de la firma del presente consentimiento informado **autoriza de manera completamente voluntaria** el uso de toda la información proporcionada durante esta entrevista y encuestas de seguimiento, así como de su información de contacto, en el desarrollo del trabajo de tesis titulado **Rediseño de los manipuladores de un robot de servicio de propósito general** elaborada por Rafael Cuéllar Ramírez y Marco Elian Soriano Pimentel, quienes fungen como investigadores principales y serán las únicas dos personas con acceso a la información recabada. El trabajo se desarrolla en el Laboratorio de Bio-Robótica adscrito al Departamento de Procesamiento de Señales de la División de Ingeniería Eléctrica de la Facultad de Ingeniería de la UNAM, como único centro de investigación participante.

El trabajo de tesis tiene como objetivo el diseño de los manipuladores y efectores finales de un robot de servicio de propósito general, así como el algoritmo de control, para permitirle tomar objetos de uso doméstico.

Como parte de la metodología de diseño centrada en el usuario de Karl T. Ulrich y Steven D. Eppingerm, se requiere conocer la problemática que se pretende resolver a través de las experiencias de los usuarios potenciales y/o de aquellos que ya hayan utilizado tecnologías similares. Con este fin se realizará una entrevista inicial con una duración aproximada de una hora y encuestas de seguimiento posteriores. Durante la entrevista se grabará audio y se tomarán notas. Las encuestas de seguimiento se enviarán por medios electrónicos.

Su participación en este trabajo de investigación no representa riesgo de lesiones o problemas de salud relacionados, por lo tanto, no se contemplan normas de indemnización. Sin embargo, su participación conlleva los riesgos previsibles y no previsibles que se listan a continuación:

- Posible incomodidad al compartir información sobre necesidades personales de asistencia doméstica.
- Fatiga durante la sesión de entrevista.
- Riesgo mínimo de identificación indirecta a pesar de la codificación de datos.

Por otro lado, aunque **no se contempla una remuneración económica**, su participación conlleva los siguientes beneficios:

- Contribución directa al desarrollo de tecnología de asistencia robótica que puede mejorar la calidad de vida de personas con necesidades similares.
- Oportunidad de reflexionar sobre sus experiencias y necesidades, lo cual puede generar aprendizajes valiosos para su propio contexto.
- Retroalimentación que puede enriquecer su perspectiva sobre el diseño centrado en el usuario.
- Participación en investigación que busca cerrar la brecha entre el desarrollo tecnológico y las necesidades reales de los usuarios finales.

Se garantizará la privacidad de los participantes mediante consentimiento informado, y la confidencialidad de la información será protegida a través de la codificación de datos, almacenamiento seguro y acceso restringido únicamente al equipo investigador. La información será utilizada con fines estrictamente académicos y de investigación y, una vez concluido el trabajo, todas las grabaciones serán eliminadas.

Al autorizar su participación, el equipo investigador se compromete a compartir con usted los resultados a través de los trabajos derivados que sean publicados.

En caso de que tenga alguna duda o inquietud relacionada con su participación, puede contactar a los investigadores principales. Adicionalmente, **puede retirarse libremente en cualquier momento sin consecuencia alguna**, mientras la investigación esté en curso mediante la **Carta de revocación de consentimiento** adjunta.

El equipo de investigación se reserva el derecho de terminar su participación sin su consentimiento en caso de que la información proporcionada no resulte relevante para el desarrollo del tema de investigación.

Este proyecto está avalado por el **Comité de Ética en Investigación y Docencia de la Facultad de Ingeniería**.

Bibliografía

[1] B. Siciliano and O. Khatib, "Robotics and the handbook," in *Handbook of Robotics* (B. Siciliano and O. Khatib, eds.), ch. 1, pp. 1–6, Springer, 2da ed., 2016.

[2] L. E. Sucar Succar, "Introducción," in *Robótica de Servicio* (E. Sucar and Y. Hernández, eds.), ch. 1, pp. 1–5, Academia Mexicana de Computación, A.C., 2da ed., 2019.

[3] J. Savage Carmona, "Robots de servicio," in *Robótica de Servicio* (E. Sucar and Y. Hernández, eds.), ch. 2, pp. 7–23, Academia Mexicana de Computación, A.C., 2da ed., 2019.

[4] T. Bajd and M. Mihelj, *Introduction to Robotics*. Springer, 1ra ed., 2013.

[5] R. C. Arkin, *Behavior-Based Robotics*. MIT Press, 1ra ed., 1998.

[6] B. Gates, "A robot in every home," in *Scientific American, Inc.*, pp. 58–65, 2006.

[7] J. Stückler and S. Behnke, "Integrating indoor mobility, object manipulation, and intuitive interaction for domestic service tasks," in *2009 9th IEEE-RAS International Conference on Humanoid Robots*, pp. 506–513, 2009.

[8] RoboCup Federation, "A brief history of RoboCup." `https://www.robocup.org/a_brief_history_of_robocup`, 2025. Accessed: 2025-10-06.

[9] RoboCup Federation, "RoboCup@Home League." `https://athome.robocup.org/`, 2025. Official website of the RoboCup@Home league for autonomous service robots.

[10] A. Bicchi and V. Kumar, "Robot grasping and contact: A review," in *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings*, pp. 348–353, 2000.

[11] M. Prats, P. J. Sanz, E. Martínez, R. Marín, and A. P. del Pobil, "Manipulación autónoma multipropósito en el robot de servicios jaume-2," in *Revista Iberoamericana de Automática e Informática Industrial*, pp. 25–37, 2008.

[12] BioRobotics Laboratory, UNAM, "Pumas OPL (Justina Robot)." [Online]. Available: `https://biorobotics.fi-p.unam.mx/robot-justina/`, 2025. Accessed: Oct. 1, 2025.

[13] C. Peralta Vázquez, "Justina, robot de servicio creado en la unam." `https://www.uv.mx/prensa/ciencia/justina-robot-de-servicio-creado-en-la-unam/`, 2019. Accessed: 2025-03-15.

[14] UNAM Global, "En 30 años, robots con inteligencia artificial, que atenderían a los humanos." `https://unamglobal.unam.mx/global_revista/en-30-anos-crearemos-robots-con-inteligencia-artificial-que-atenderian-a-los-hum` 2017. Accessed: 2026-08-19.

[15] J. Savage, A. Llarena, G. Carrera, S. Cuellar, D. Esparza, Y. Minami, and U. Peñuelas, *ViRbot: A System for the Operation of Mobile Robots*, vol. 5001, pp. 512–519. 05 2007.

[16] M. A. Negrete Villanueva, *Sistema de navegación para un robot de servicio*. Tesis de doctorado, Universidad Nacional Autónoma de México, Posgrado en Ciencia e Ingeniería de la Computación, Ciudad de México, México, 2019.

[17] Open Robotics, "About ROS."

[18] Open Robotics, "ROS 2 Concepts: Nodes." `https://docs.ros.org/en/ros2_documentation/Concepts/Basic/About-Nodes.html`. Accessed: 2026-08-11.

[19] B. Gerkey, "Why ROS 2?." `https://design.ros2.org/articles/why_ros2.html`, June 2014. Last modified: May 2022.

[20] Open Robotics, "ROS 2 Concepts: Topics, Services, and Actions."

[21] J. J. Craig, *Introduction to Robotics: Mechanics and Control*. Upper Saddle River, NJ: Pearson Prentice Hall, 3 ed., 2005.

[22] R. L. Norton, *Diseño de maquinaria: Síntesis y análisis de máquinas y mecanismos*. México, D.F.: McGraw-Hill Interamericana, 5 ed., 2013.

[23] F. Reuleaux, *The Kinematics of Machinery*. New York: Dover Publications, 1963.

[24] J. J. Uicker, G. R. Pennock, and J. E. Shigley, *Theory of Machines and Mechanisms*. New York: Oxford University Press, 3 ed., 2003.

[25] T. Kivelä, *Increasing the Automation Level of Serial Robotic Manipulators with Optimal Design and Collision-free Path Control*. Doctoral dissertation, Tampere University of Technology, Tampere, Finland, 2017.

[26] K. J. Waldron and J. Schmiedeler, "Kinematics," in *Springer Handbook of Robotics* (B. Siciliano and O. Khatib, eds.), pp. 11–36, Springer, 2 ed., 2016.

[27] Open Robotics, "Quaternion fundamentals." `https://docs.ros.org/en/jazzy/Tutorials/Intermediate/Tf2/Quaternion-Fundamentals.html`, 2026. ROS 2 Jazzy documentation. Accessed: 2026-08-30.

[28] J. Haviland and P. Corke, "Manipulator differential kinematics: Part ii: Acceleration and advanced applications," *IEEE Robotics & Automation Magazine*, pp. 2–12, 2023.

[29] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. London: Springer, 2009.

[30] M. W. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*. Hoboken, NJ: Wiley, 2 ed., 2020.

[31] H. I. Christensen and G. D. Hager, "Sensing and estimation," in *Springer Handbook of Robotics* (B. Siciliano and O. Khatib, eds.), pp. 91–112, Springer, 2 ed., 2016.

[32] W. Chung, L.-C. Fu, and S.-H. Hsu, "Motion control," in *Springer Handbook of Robotics* (B. Siciliano and O. Khatib, eds.), pp. 163–194, Springer, 2 ed., 2016.

[33] L. Villani and J. De Schutter, "Force control," in *Springer Handbook of Robotics* (B. Siciliano and O. Khatib, eds.), pp. 195–220, Springer, 2 ed., 2016.

[34] N. Hogan, "Impedance control: An approach to manipulation: Part I—theory," *Journal of Dynamic Systems, Measurement, and Control*, vol. 107, no. 1, pp. 1–7, 1985.

[35] M. H. Raibert and J. J. Craig, "Hybrid position/force control of manipulators," *Journal of Dynamic Systems, Measurement, and Control*, vol. 103, no. 2, pp. 126–133, 1981.

[36] G. Pahl, W. Beitz, J. Feldhusen, and K.-H. Grote, *Engineering Design: A Systematic Approach*. London: Springer-Verlag, 3rd ed., 2007. Traducción al inglés de la obra original en alemán *Konstruktionslehre*.

[37] M. Garreta Domingo and E. Mor Pera, *Diseño centrado en el usuario*. Barcelona: Fundació per a la Universitat Oberta de Catalunya, 2013. Material didáctico UOC. PID_00176058.

[38] D. A. Norman, *The Design of Everyday Things*. New York: Basic Books, revised and expanded ed., 2013. Primera edición publicada en 1988 como *The Psychology of Everyday Things*.

[39] T. Lowdermilk, *User-Centered Design*. Sebastopol, CA: O'Reilly Media, 2013.

[40] RoboCup@Home Technical Committee, "Robocup@home." `https://athome.robocup.org/`, 2024. Accessed: 2025-03-15.

[41] K. T. Ulrich and S. D. Eppinger, *Diseño y desarrollo de productos*. México, D.F.: McGraw-Hill/Interamericana Editores, S.A. de C.V., 5 ed., 2013. Traducción de la 5a. edición de *Product Design and Development*. Traducción: Jorge Humberto Romo Muñoz y Ricardo Martín Rubio Ruiz.